

Poglavje 17 **P10** Objektni razvoj, simulacija razvoja informacijskega sistema za bankomat in simulacija projekta



P10

17.1 Objektni razvoj

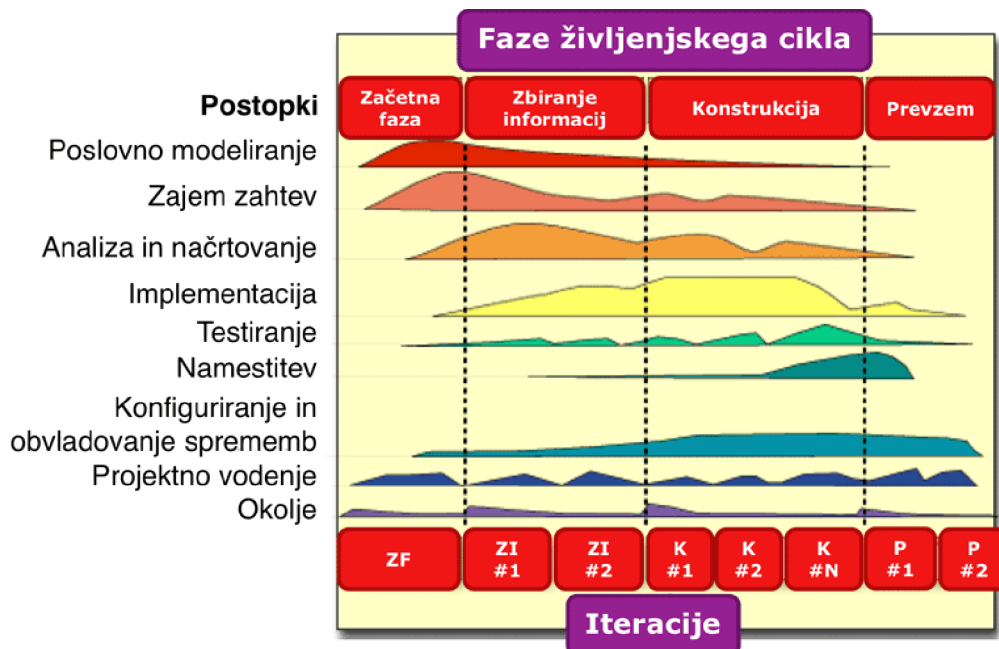
17.1.1 RUP proces razvoja

Objektni proces razvoja je skupinski pristop k razvoju, ki temelji na poenotenem procesu (angl. *W Rational Unified Process (RUP)*) in jeziku za modeliranje (angl. *W Unified Modeling Language (UML)*), kot prikazuje slika 17.1.



Slika 17.1: Objektni razvoj

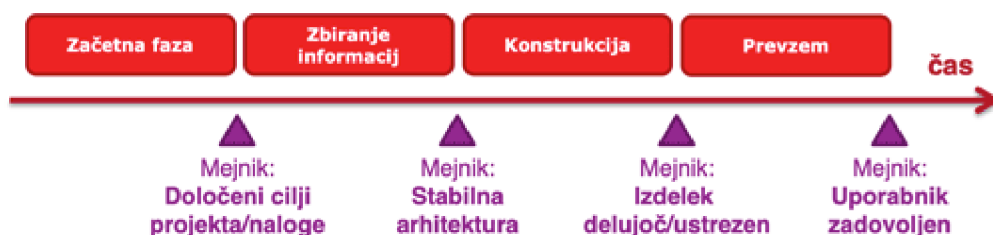
RUP je primer procesa objektnega razvoja, katerega avtor je podjetje *W Rational Software* in ima opredeljuje faze življenjskega cikla, ki se izvajajo v iteracijah, kot to prikazuje slika 17.2.



Slika 17.2: Faze življenjskega cikla, iteracije in postopki RUP procesa razvoja

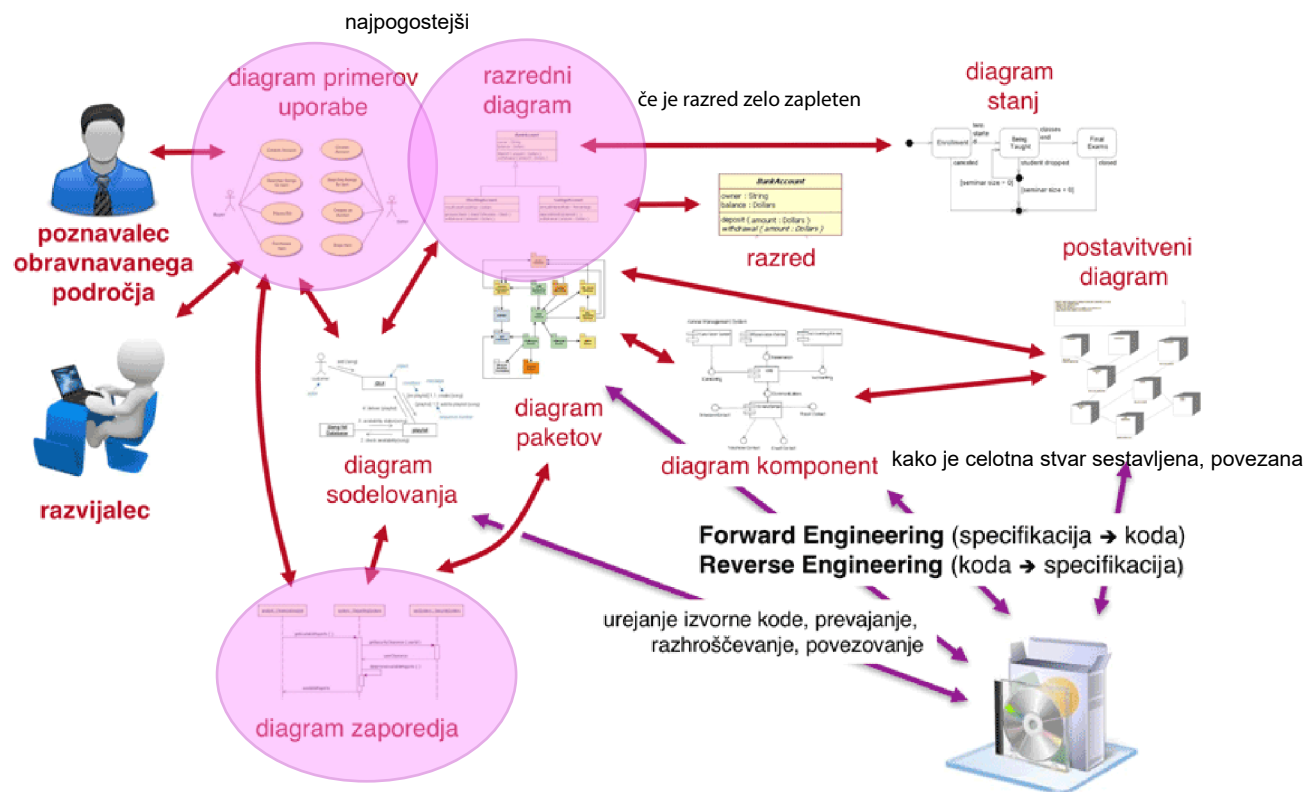
RUP zajema **4 faze življenjskega cikla** (glej slike 17.2 in 17.3):

- **začetna faza** - vzpostavitev projekta, opredelitev okvirjev obravnavanega področja, načrtovanje virov,
- **zbiranje informacij** - zbiranje informacij o obravnavanem področju, specifikacija značilnosti, jasen načrt kako se bomo zadeve lotili, načrtovanje arhitekture,
- **konstrukcija** - konstrukcija izdelka,
- **prevzem** - predaja izdelka v uporabo končnemu uporabniku.



Slika 17.3: Faze življenjskega cikla in mejniki

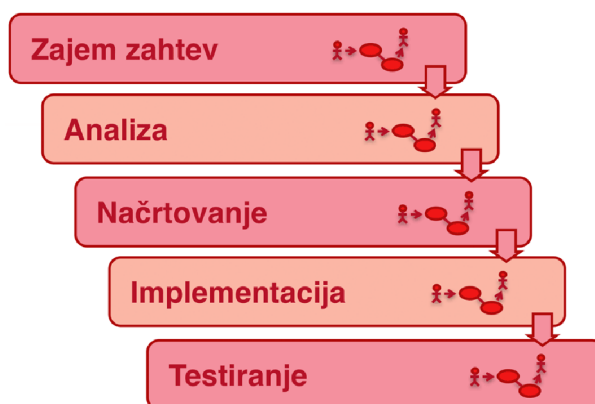
RUP predvideva uporabo različnih UML diagramskih tehnik za specifične potrebe, kot to prikazuje slika 17.4.



Glavne lastnosti RUP procesa razvoja:

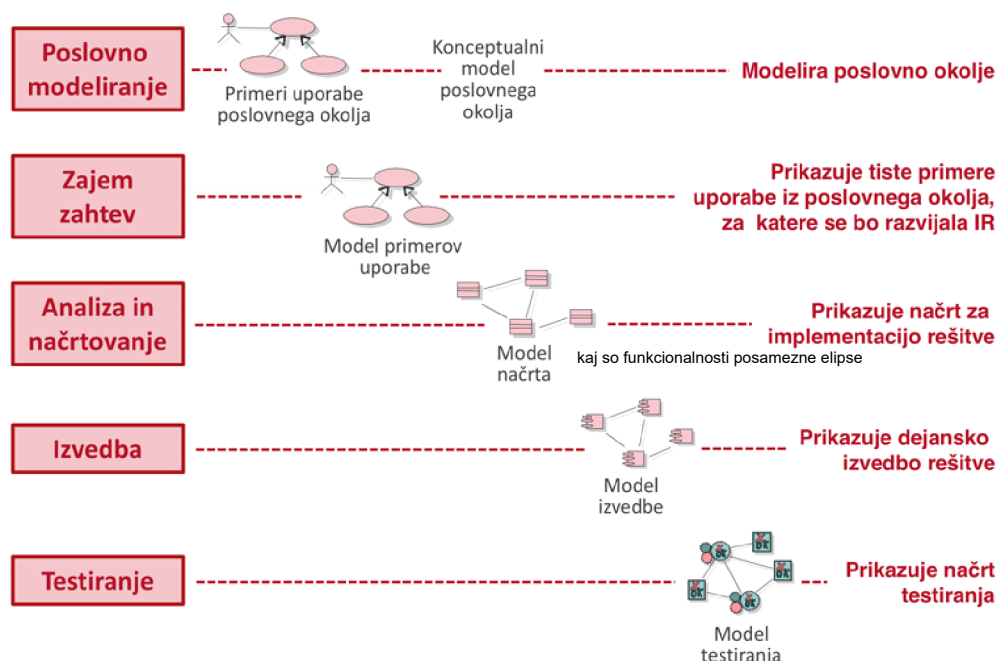
- daje **smernice za učinkovit razvoj** kakovostne programske opreme,
- **zmanjšuje tveganja** in povečuje predvidljivost,
- zajema in **vpeljuje najboljše izkušnje**,
 - učenje iz izkušenj drugih,
 - mentorstvo v elektronski obliki,
 - razširitev izobraževalnega gradiva,
- pospešuje **skupno vizijo in kulturo**,
- vpeljuje **načrt za uvedbo orodij**,
- omogoča enostaven in hiter dostop do informacij v elektronski obliki (npr. spletna stran ipd.).

Primeri uporabe (angl. [W Use cases](#)) so hrbtenica procesa RUP (glej sliko 17.5). najprej naredimo diagram primrov uporabe



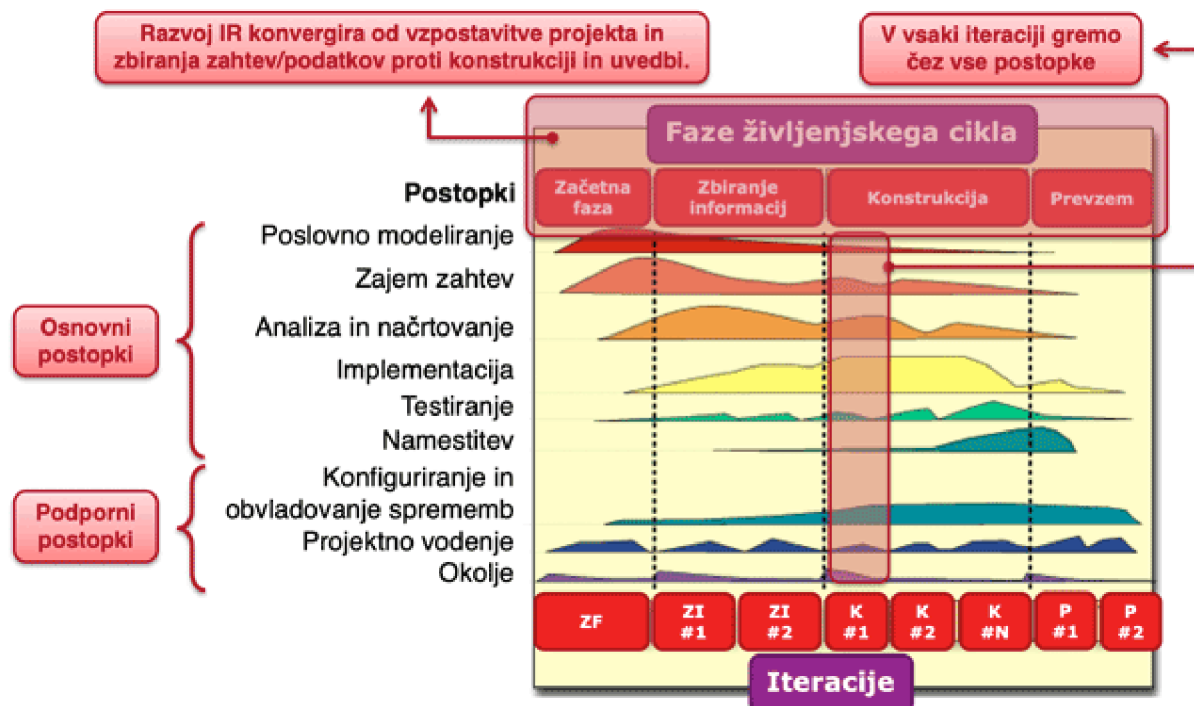
Slika 17.5: Vloga primerov uporabe

Pri gradnji **modela sistema** imajo primeri uporabe zelo pomembno vlogo, saj na podlagi njih začnemo gradnjo modela sistema po posameznih fazah življenjskega cikla (glej sliko 17.6).



Slika 17.6: Gradnja modela sistema

Prednosti arhitekturno usmerjenega pristopa RUP so prikazane na sliki 17.7.



Slika 17.7: Prednosti RUP procesa razvoja

17.1.1.1 Zajem zahtev



Slika 17.8: **Zajem zahtev** po fazah življenjskega cikla

Namen zajema zahtev je:

- doseči **soglasje s stranko** oz. uporabniki, kaj naj sistem dela,
- omogočiti **razvijalcem boljše razumevanje zahtev** sistema,

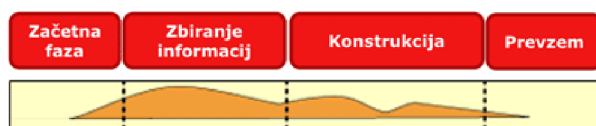
- določiti **meje sistema**,
- zagotoviti **osnovo za načrtovanje tehnične vsebine**,
- definirati **grafični uporabniški vmesnik** (angl. *W Graphical User Interface (GUI)*) sistema.

Namen zajema zahtev je podoben kot pri strukturnem pristopu. V določeni meri uporabljamo tudi podobne tehnike kot pri strukturnem pristopu.

RUP za zapis zahtev uvaža **višjo stopnjo formalnosti** (glej sliko 17.4):

- **primeri uporabe** (jezik UML),
- opis s **tokovi dogodkov**,
- uporaba drugih tehnik UML v posebnih primerih (npr. diagrami stanj, diagrami aktivnosti, diagrami interakcije).

17.1.1.2 Analiza in načrtovanje



Slika 17.9: **Analiza in načrtovanje** po fazah življenjskega cikla

Namen analize in načrtovanja je:

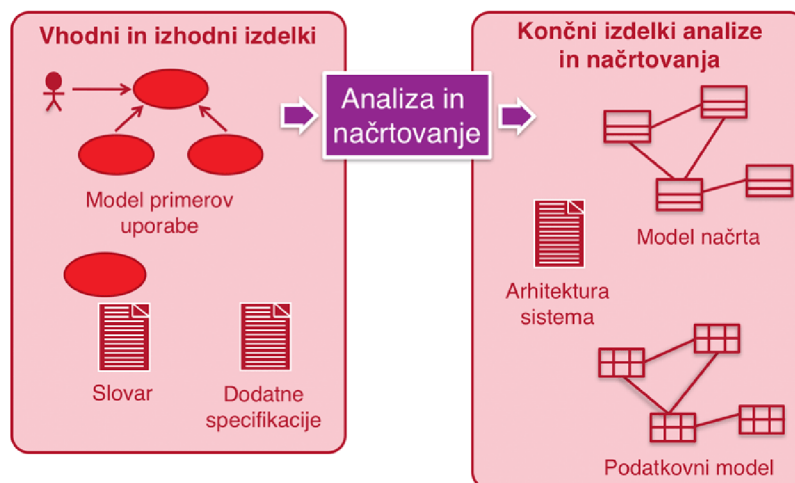
- pretvoriti **zahteve v načrt sistema**,
- razviti robustno **arhitekturo sistema**,
- **prilagoditi** načrt **izvedbenemu okolju**.

Povezava z drugimi postopki:

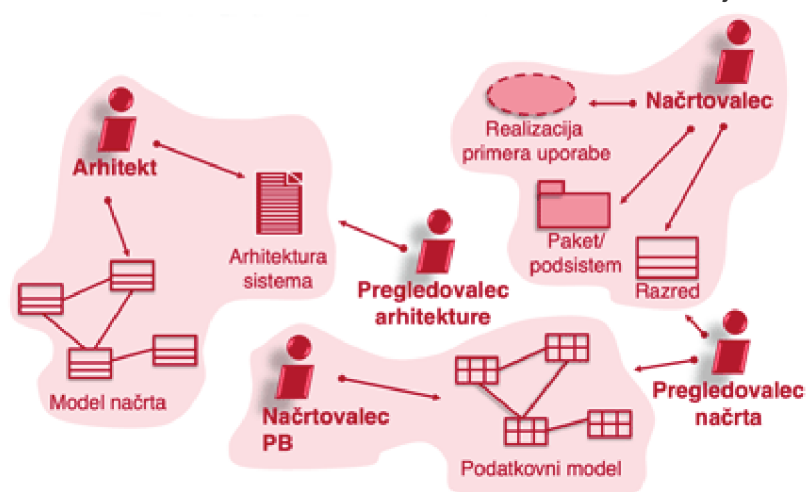
- **poslovno modeliranje** postavi **organizacijski kontekst sistema**,
- postopek **zajema zahtev** je primarni vhod v postopek analize in načrtovanja,
- postopek **izvedbe** realizira sistem na osnovi načrta.

Tabela 17.1: **Primerjava** objektne analize in načrtovanja

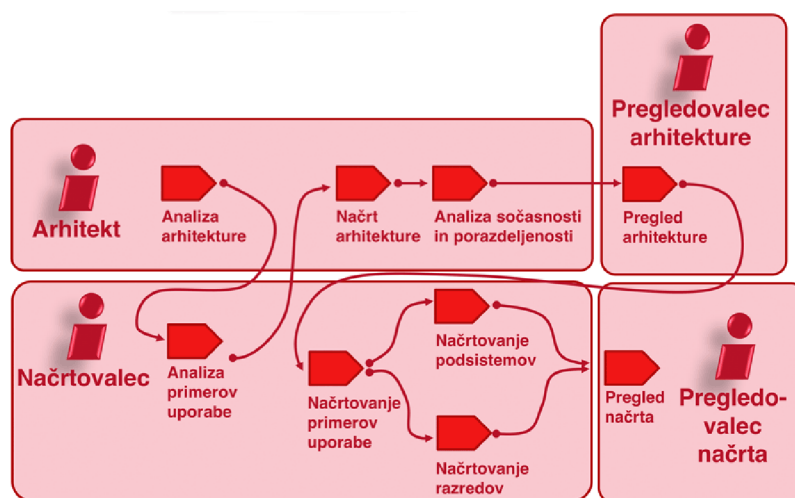
Analiza	Načrtovanje
Osredotoča se na razumevanje problema , idealizirano načrtovanje, obnašanje, strukturo sistema, funkcionalne zahteve, majhen model.	Osredotoča se na razumevanje rešitve , operacije in attribute, zmogljivost rešitve, blizu programski kodi, življenjski cikel objektov, nefunkcionalne zahteve, velik model.



Slika 17.10: **Vhodi in izhodi** iz analize in načrtovanja



Slika 17.11: **Izdelki** iz analize in načrtovanja



Slika 17.12: **Postopki** iz analize in načrtovanja

17.1.1.3 Univerzalni modelirni jezik UML

Univerzalni modelirni jezik (UML) je jezik za specifikacijo, vizualizacijo, konstrukcijo in dokumentacijo izdelkov v okviru objektnega razvoja informacijskih rešitev.

V začetku 90-ih več kot 50 objektno usmerjenih (OO) metod.

- Fusion, Shlaer-Mellor, ROOM, Wirfs-Brock, Coad-Yourdon, MOSES, BOOM, OOSD, OSA, BON, Catalysis, HOOD, Ooram, DOORS idr.

Problemi:

- presek in konflikti v meta-modelih,
- različne grafične notacije,
- procesi so različni ali manjkajo,
- gospodarstvo potrebuje standard!



Slika 17.13: UML diagrami s statičnega in dinamičnega vidika

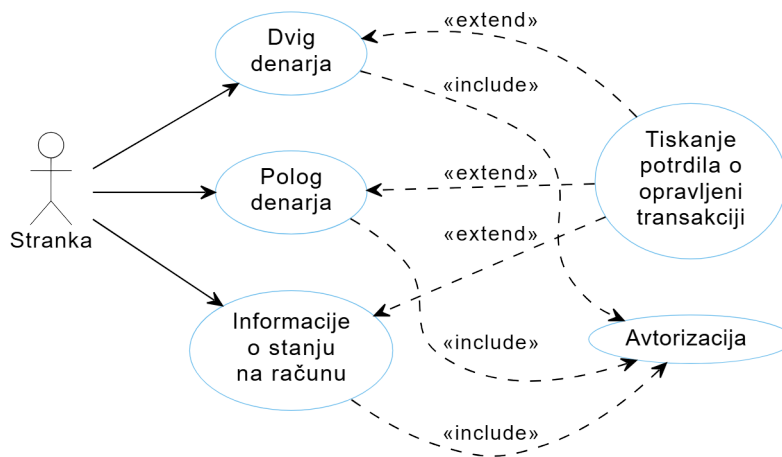
17.1.1.3.1 Diagrami primerov uporabe

Diagrami primerov uporabe (DPU) (angl. *Use Case Diagrams (UCD)*) predstavljajo **povezavo med akterji** (uporabniki sistema) in procesi, ki v sistemu tečejo. Namen uporabe DPU je **predstavitev in zajemanje uporabniških zahtev**, s katerimi modeliramo tudi vplive okolja na sistem. Sistem modeliramo kot množico procesov, ki uporabniku **ponujajo usluge sistema**, po drugi strani je to množica akterjev, ki vplivajo na sistem.

Gradniki:

- **primer uporabe** (angl. *use case*),
- **akter** (angl. *actor*) in
- **povezave** med primeri uporabe in akterji.

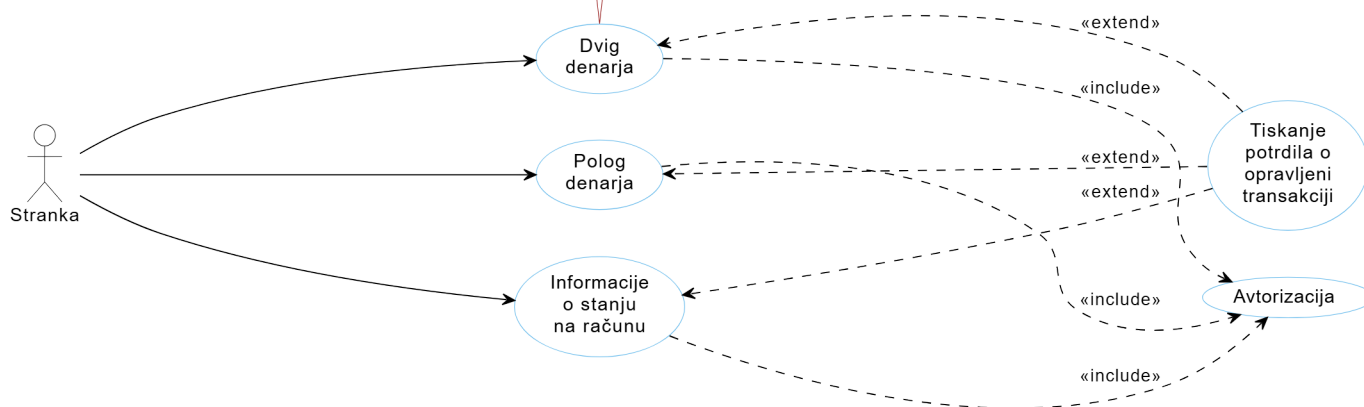
Primer uporabe opisuje posamezen tok dogodkov skozi sistem, ki akterju **vrne merljiv rezultat** (npr. dvig denarja na bankomatu na sliki 17.14).



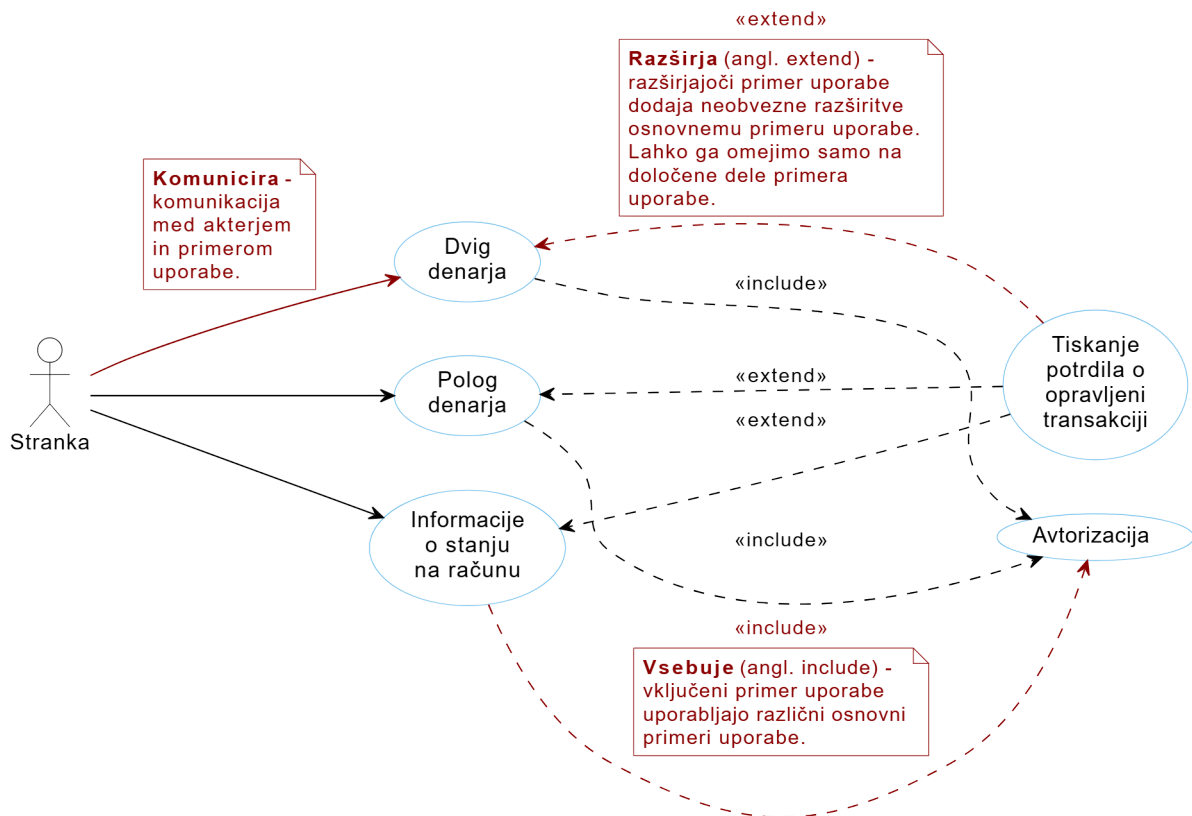
Slika 17.14: Primer diagrama primerov uporabe za **dvig denarja na bankomatu** PlantUML

Scenarij za primer uporabe: **Dvig denarja**

- A. Stranka želi dvigniti denar.
- B. Stranka vstavi kartico in bankomat preveri ali je kartica nepoškodovana.
- C. Bankomat preveri ali je kartica veljavna (rok veljavnosti).
- D. Stranka vnese geslo.
- E. Bankomat preveri pravilnost gesla.
- F. Bankomat preveri ali je stanje na računu pozitivno.
- G. Stranka izbere dvig denarja in vnese željeno vsoto.
- H. Bankomat preveri ali stranka lahko dvigne željeno vsoto glede na dnevni limit in stanje na računu.
- I. Stranka želi izpis potrdila o opravljeni transakciji.
- J. Bankomat izvede dvig denarja iz računa, natisne potrdilo in izplača denar stranki.
- K. Primer uporabe je zaključen.



Slika 17.15: Opis scenarija za primer uporabe **Dvig denarja** PlantUML



Slika 17.16: Opis posameznih gradnikov na primeru uporabe PlantUML

Vrste primerov uporabe:

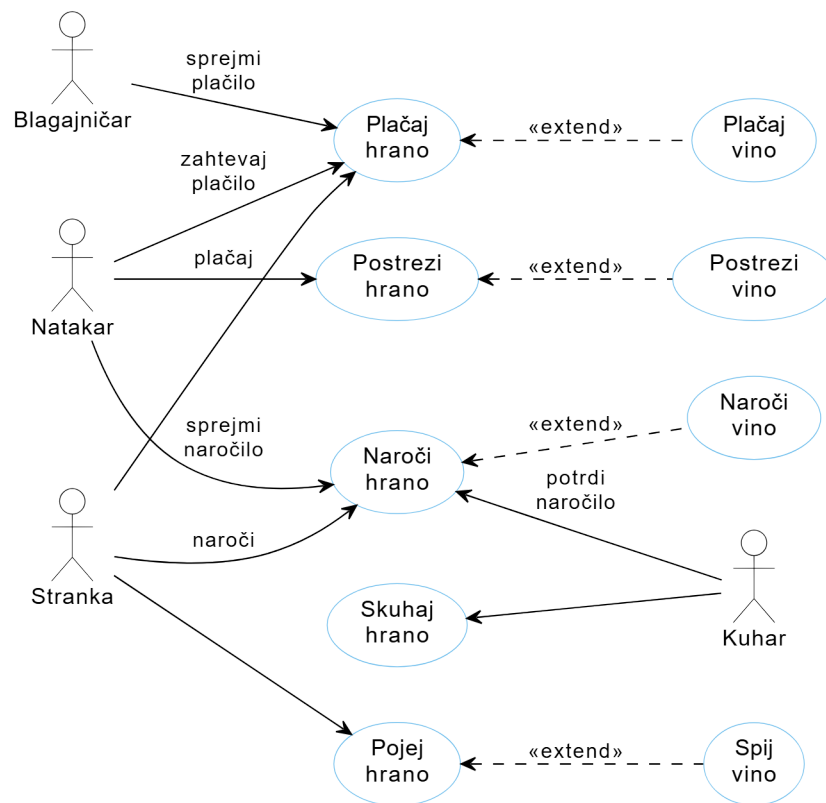
- **poslovni** primer uporabe predstavlja odziv poslovnega sistema na dogodke,
- **sistemski** primer uporabe predstavlja interakcijo s programsko opremo.

Osnovni primeri uporabe se razdelijo na več scenarijev in variant. Primeri uporabe se nanašajo na zunanji pogled na sistem in nikakor ne na razrede znotraj sistema.

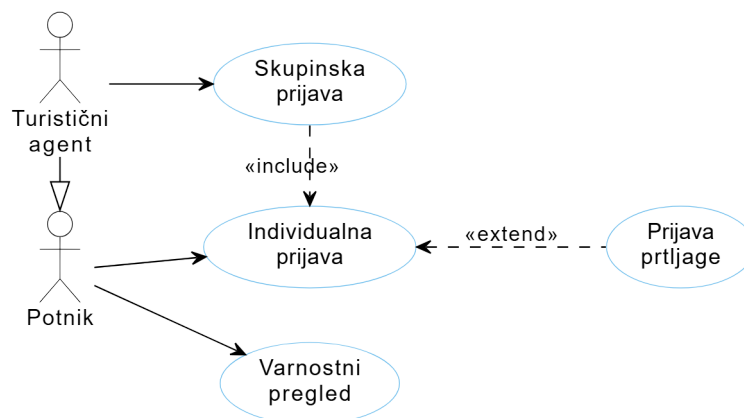
Opomba: Ena izmed najpogostejših napak pri načrtovanju diagramov primerov uporabe je modeliranje interne podatkovne baze kot akter!

Akterji se vedno nahajajo zunaj meja našega sistema!

17.1.1.3.1.1 Primeri



Slika 17.17: Primer naročanja v restavraciji PlantUML



Slika 17.18: Primer prijave na let PlantUML

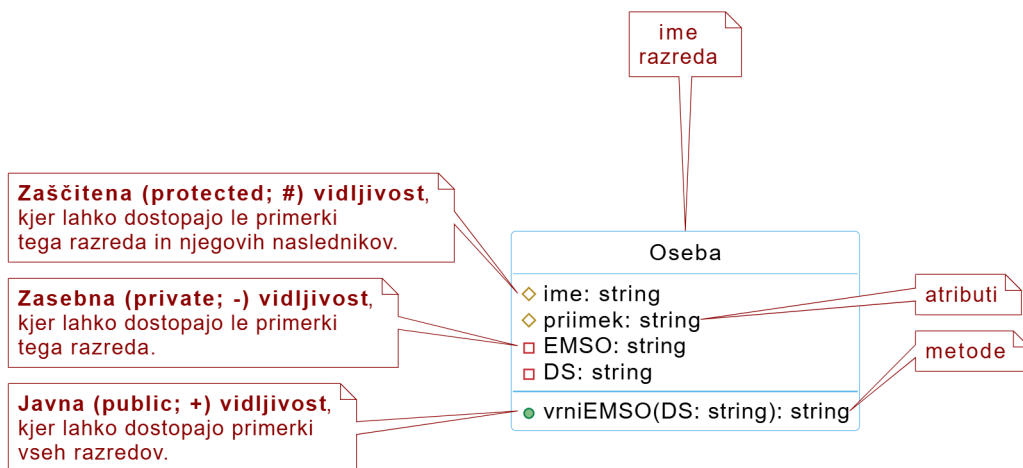
17.1.1.3.2 Razredni diagrami

Razredni diagrami prikazujejo statično strukturo modela sistema, in sicer:

- razrede,
- strukturo razredov,
- metode,
- attribute in
- relacije med razredi.

Diagram objektov prikazuje posnetek stanja sistema v določenem trenutku.

Razred predstavlja množico objektov, ki imajo podobno strukturo, obnašanje in relacije.



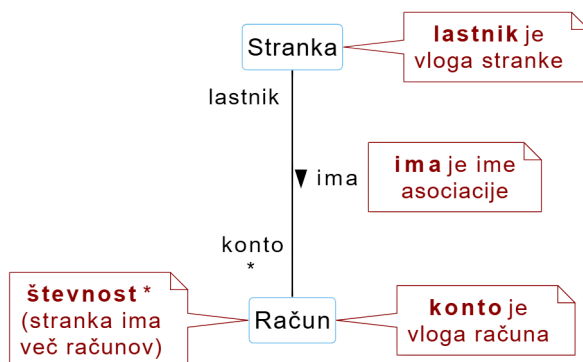
Slika 17.19: Lastnosti asociacije PlantUML

Med razredi je možnih **več vrst relacij**:

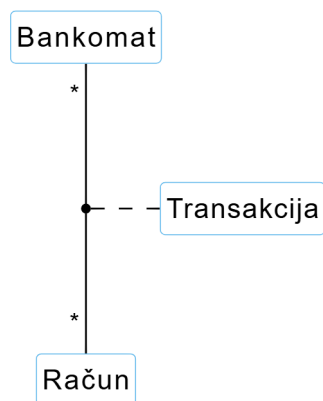
- **asociacija** (angl. *association*),
- **agregacija** (angl. *aggregation*) in **kompozicija** (angl. *composition*) kot posebna primerka asociacije,
- relacija **dedovanja** (angl. *generalization, is-a*) in
- **odvisnost** (angl. *dependency*) med elementi.

Asociacija predstavlja **statično povezavo med razredi**. Poleg vlog lahko asociacija vsebuje še (glej sliko 17.20):

- **števnost**,
- **urejenost** (angl. *ordering*),
- **določilo** (angl. *qualifier*) in
- **navigacijo** (angl. *navigability*).



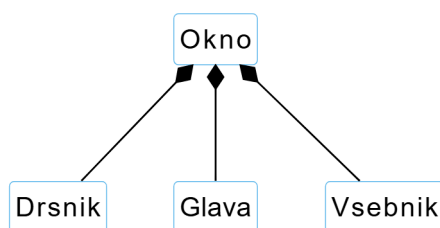
Slika 17.20: Lastnosti asociacije PlantUML



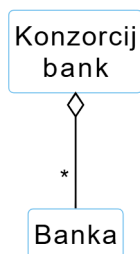
Slika 17.21: **Asociacija kot razred** ima lastnosti razreda. PlantUML

Agregacija in **kompozicija** sta asociaciji, ki predstavljata relacijo **“je del nečesa”** (angl. *part of*).

- **Kompozicija** je posebna vrsta agregacije z **močnim lastništvom** (glej primer na sliki 17.22).
- Pri **agregaciji** (glej primer na sliki 17.23) del **lahko pripada več lastnikom** (agregatom), pri **kompoziciji** pa to ni dovoljeno.

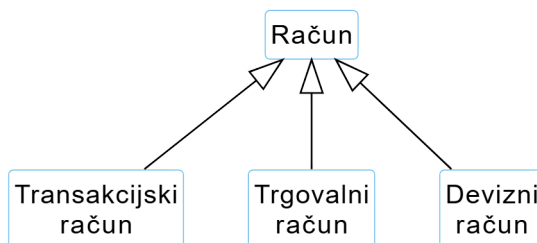


Slika 17.22: Primer kompozicije PlantUML



Slika 17.23: Primer agregacije PlantUML

Dedovanje je realizirano kot **generalizacija/specializacija**, ki je relacija med splošnim in specifičnim elementom (glej primer na sliki 17.24).

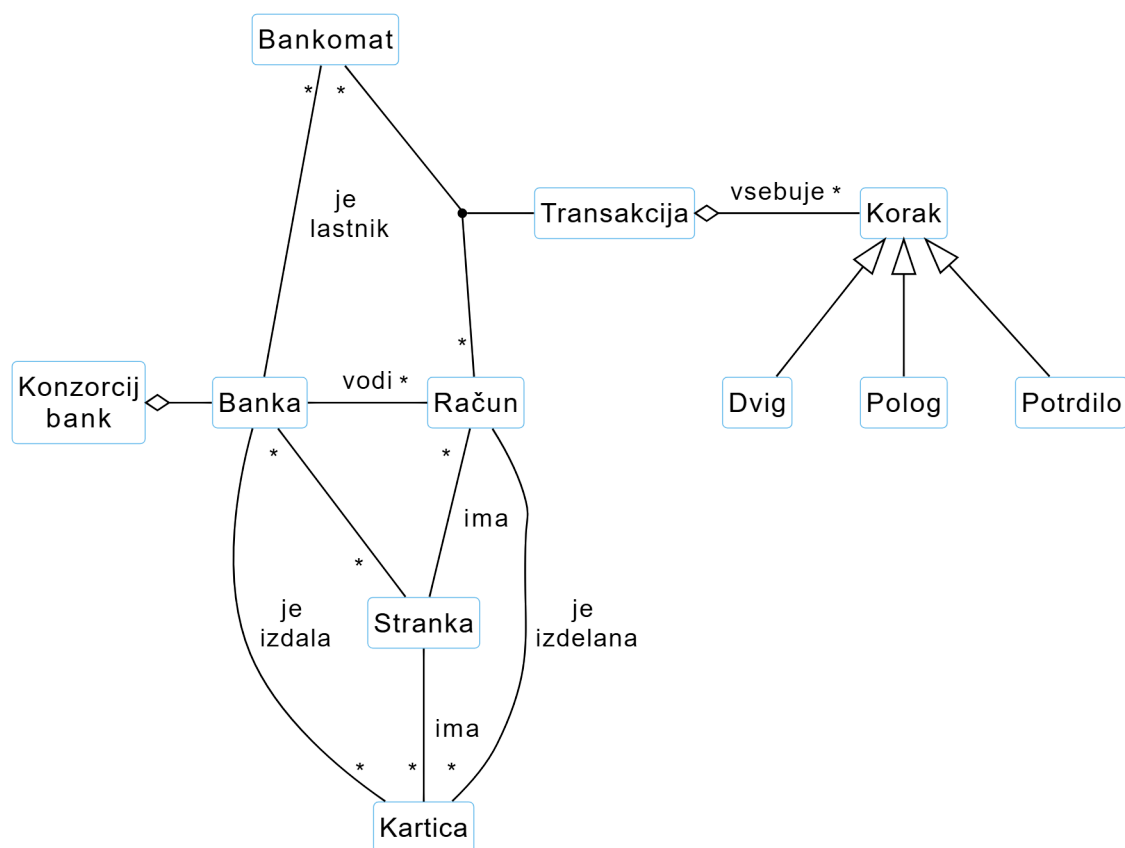


Slika 17.24: Primer dedovanja PlantUML

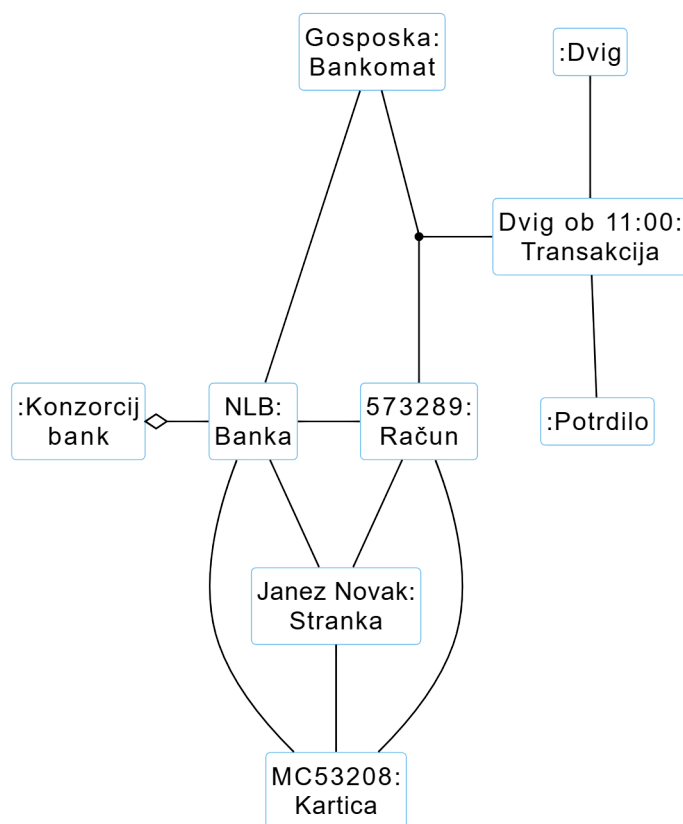
Razredne diagrame uporabljamo **za modeliranje**:

- **poslovnih konceptov in določitev mej sistema,**
- **preprostega sodelovanja,**
- **logične sheme podatkovne baze.**

17.1.1.3.2.1 Primeri

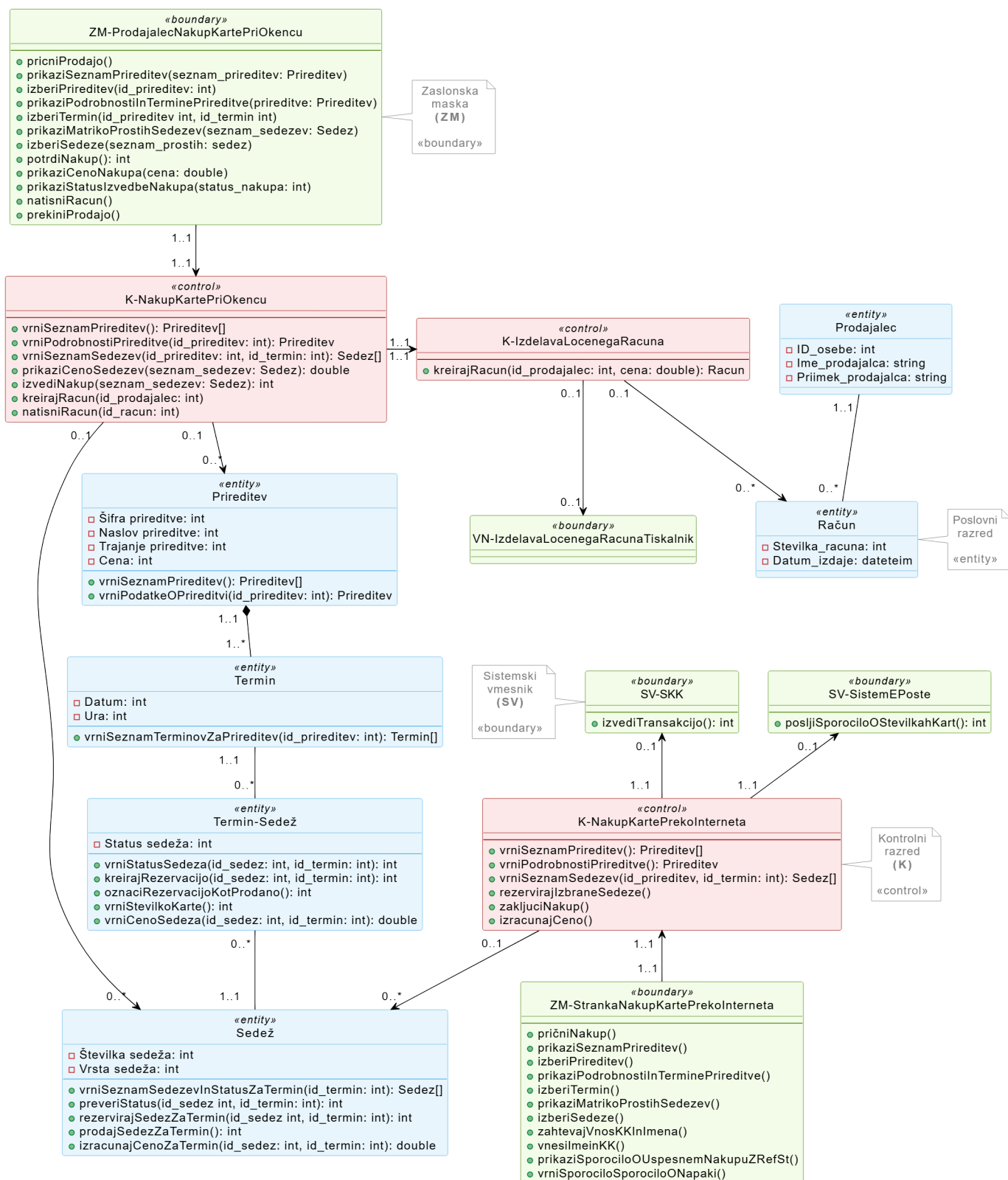


Slika 17.25: **Razredni diagram** za sistem bankomatov PlantUML



Slika 17.26: **Objektni diagram** za sistem bankomatov PlantUML

Slika 17.27 prikazuje organizacijo razredov v razrednem diagramu z vsemi sodelujočimi razredi (*angl. View of Participating Classes (VOPC)*).



Slika 17.27: Razredni VOPC diagram (z mejnimi, kontrolnimi in poslovnimi razredi) za nakup vstopnic

PlantUML

17.1.1.3.3 Diagram komponent

Diagram komponent (angl. *Component diagram*) prikazuje odvisnosti med programskimi komponentami. V osnovi so to razredni diagrami, ki obravnavajo sistem z vidika komponent (glej primer na sliki 17.28).



Slika 17.28: Primer **diagrama izvedljivih komponent** PlantUML

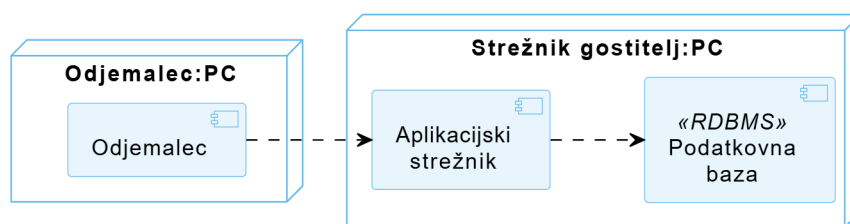
17.1.1.3.4 Postavitveni diagram

Postavitveni diagram (angl. *Deployment diagram*) uporabljamo predvsem za:

- popolnoma porazdeljene sisteme,
- sisteme tipa odjemalec/strežnik,
- vgradne (angl. *embedded*) sisteme.

Za modeliranje **popolnoma porazdeljenih sistemov** (glej primer na sliki 17.29):

- identificiramo enote v sistemu in procesorje,
- vse enote modeliramo na takšni ravni podrobnosti, kot je to potrebno za določitev optimalnih zmogljivosti,
- pozornost posvetimo logičnemu združevanju vozlišč.



Slika 17.29: Primer **postavitvenega diagrama porazdeljenega sistema** PlantUML

17.1.1.3.5 Diagram zaporedja

UML diagrami zaporedja (angl. *UML sequence diagrams*) so še ena izmed pomembnih načrtovalskih tehnik. Ponavadi se uporablja pri načrtovanju, preden razvojna ekipa začne s programiranjem. Diagrami zaporedja se uporabljajo za prikaz, kako objekti v programu medsebojno sodelujejo pri izvedbi določenih nalog. Diagram zaporedja predstavlja zemljevid komunikacije med različnimi akterji, kjer so označena sporočila, poslana od akterja do akterja.

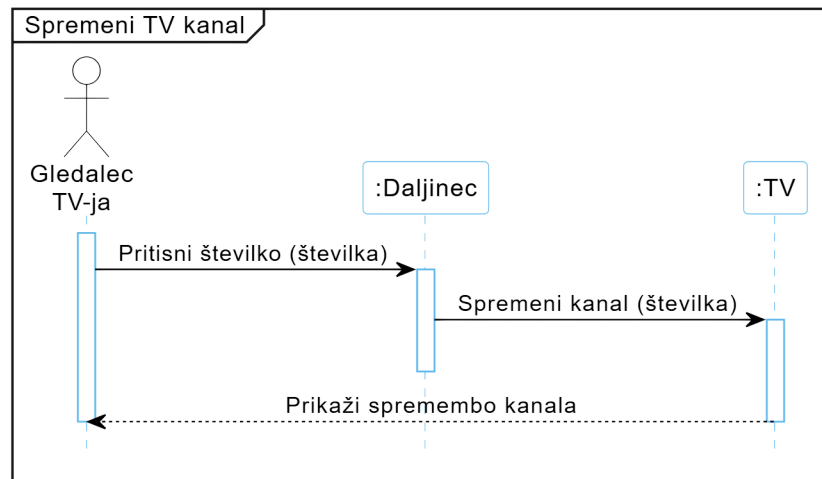
Diagram zaporedja pomaga vizualizirati razrede, ki jih sistem kreira ter funkcije, ki jih bomo morali implementirati. Takšen pogled lahko tudi ponazori morebitne težave v sistemu, ki prej niso bile znane.

Poglejmo si posamezne komponente diagrama zaporedja:

- Škatle predstavljajo vloge, ki jih imajo posamezni objekti. **Vloga** (angl. *Role*) je običajno poimenovana po razredu objekta.
- **Življenjske črte** (angl. *lifelines*) so navpične črčkane črte in se v diagramu uporabljajo za označevanje objekta v času.
- Puščice s polno črto se uporabljajo za prikaz **sporočil** (angl. *messages*), ki so poslana od enega objekta do drugega ali od pošiljatelja do prejemnika. Sprejemniki so koncu usmerjene puščice.
- Puščice s črčkano črto se uporabljajo za prikaz **vrnitve podatkov in nadzora nazaj** začetnim objektom. Kratek opis vrnitve podatkov ali nadzor je vključen v obliki besedila nad puščico.

- Majhni pravokotniki vzdolž življenjske črte predmeta označujejo **aktivacijo metode**. Objekt aktiviramo vedno, ko objekt pošlje, prejme ali čaka na sporočilo.
- V diagram zaporedja so lahko vključeni tudi **akterji** (*angl. actors*), če uporabljajo objekte ali z njimi **komunicirajo**. Akterji so ponavadi prikazani z ikono človečka. Akterji so ponavadi prikazani z ikono človečka.

Diagram zaporedja ponavadi **uokvirimo**, kjer je naslov diagrama naveden v zgornjem levem kotu (glej sliko 17.30). Dobra praksa je, da zagotovimo smiseln naslov, saj se bo diagram uporabljal kot referenca pri razvoju. Druga dobra praksa je risanje objektov od leve proti desni v zaporedju medsebojnega sodelovanja.

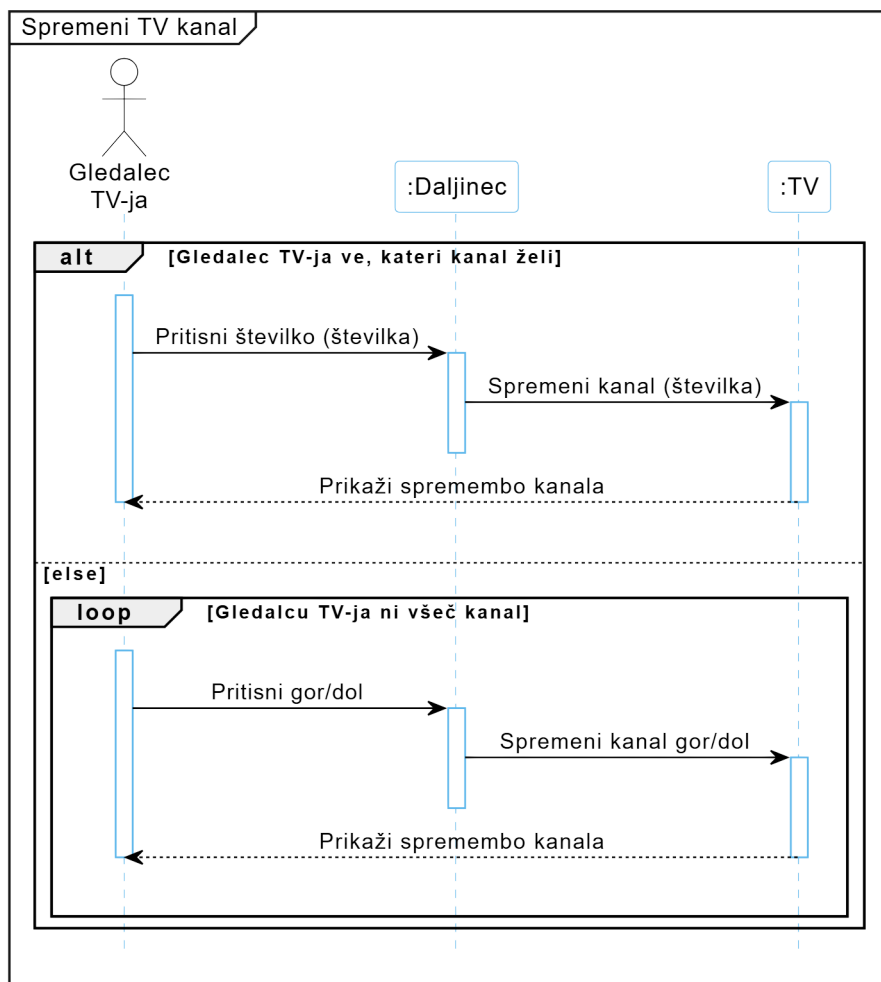


Slika 17.30: Diagram zaporedja spremembe TV kanala PlantUML

Diagram zaporedja lahko vsebuje tudi druge diagrame zaporedja. Npr. če pripravljamo diagram zaporedja za bankomat, lahko obstaja drugačno zaporedje za dvige in pologe, med enim postopkom bo nekdo morda želel dvigniti in položiti denar. V našem diagramu zaporedja bi imeli eno veliko **zaporedje aktivnosti** z dvema manjšima zaporedjema znotraj njih.

Ko načrtujemo programsko opremo, lahko diagrami zaporedja postanejo veliko bolj zapleteni. V diagramu zaporedja lahko prikažemo tudi zanke in alternativne poti. **Alternativni proces** se bo izvedel, ko bo **izpolnjen določen pogoj**. Alternativno izvajanje lahko postavimo v polje in ga v zgornjem desnem kotu označimo z **alt**.

Za ponazoritev tega lahko nadgradimo prejšnji primer na sliki 17.30. Predstavljajmo si, ko gledalec TV-ja ni prepričan, na kateri kanal bi želel preklopiti in bi preprosto brskal med kanali, dokler ne izbere tistega, ki mu je všeč. Ta scenarij lahko ponazorimo z vključitvijo pogoja `[else]`. To pomeni, da se scenarij zgodi le, če so vse druge možnosti neresnične.



Slika 17.31: Diagram zaporedja spremembe TV kanala z izjemami PlantUML

Diagrami zaporedja so uporabna tehnika pri pripravi čistih in dobro zasnovanih programov.

17.1.1.3.6 Diagram stanj

UML diagrami stanj (angl. *UML state diagrams*) so druga vrsta diagramov, ki se uporablja za opis, kako se sistemi obnašajo in odzivajo. Sledijo stanjem sistema ali posameznega objekta in prikazujejo spremembe med stanji, ko se v sistemu zgodi vrsta dogodkov.

Diagram stanja ponazarja vedenja objekta s prikazom spreminjajočih se stanj objekta. Te spremembe so odziv na različne dogodke. **Stanje** (angl. *State*) je način, kako objekt obstaja v določenem trenutku. Stanje objekta je določeno z vrednostmi njegovih atributov.

Dobra analogija stanjem je primer avta. Avto s samodejnim menjalnikom ima lahko različna stanja: parkiraj, vzvratno, nevtrarno in vožnja. Če je avto v stanju vzvratno, se obnaša tako, da se premika nazaj. Če se želite premakniti naprej, morate spremeniti stanje avta v vožnja. To je podobno stanju objektov v sistemu. Ko je objekt v določenem stanju, se obnaša na določen način ali ima attribute nastavljene na določene vrednosti. Elementi diagrama stanja so:

- Poln krog označuje začetno stanje objekta. Vsak diagram stanja se začne s polnim krogom.
- Zaobljen pravokotnik označuje druga stanja. Ti pravokotniki imajo tri razdelke: ime stanja, spremenljivke stanja in aktivnosti.
 - Imena stanj morajo biti kratki, smiselni naslovi stanj objekta. Vsako stanje mora imeti vsa ime stanja.

- Spremenljivke stanja so podatki, pomembni za stanje objekta.
- **Aktivnosti** (*angl. Activities*) so akcije, ki se izvajajo v določenem stanju. Za vsako stanje poznamo tri vrste aktivnosti: **Entry**, **Exit** in **Do**. **Aktivnosti Entry** so tiste, ki se zgodijo, ko se v stanje vstopi iz drugega stanja. **Aktivnosti Exit** so tiste, ki se zgodijo, ko se stanje zapusti in se zgodi prehod v drugo stanje. **Aktivnosti Do** se zgodijo, ko je predmet v določenem stanju.
- Puščice označujejo **prehode** (*angl. transitions*) iz enega stanja v drugo. Prehode običajno sproži dogodek, ki je zapisan nad puščico.
 - Vsaka puščica prehoda ima vedno opredeljen dogodek, prisoten pa je lahko tudi pogoj in akcija. Prehod in akcija se zgodi iz danega stanja, če se dogodek zgodi in je pogoj resničen.
- Krog z zapolnjenim notranjim krogom označuje konec, ki predstavlja uničenje objekta ali zaključek procesa. Vsi diagrami nimajo zaključka, saj lahko delujejo neprekinjeno.

Diagrami stanj so lahko koristni pri določanju dogodkov, ki se lahko zgodijo med življenjsko dobo objekta, kot so npr. različni uporabniški vnosi, in kako naj se objekt obnaša, ko do teh dogodkov pride, kot je preverjanje pogojev in izvajanje akcij. Včasih je morda lažje videti spremembo stanja v diagramu, namesto branja izvirne kode.

Diagrami stanj lahko pomagajo tudi prepoznavati težave v sistemu, kot je odkrivanje stanj, ki niso načrtovana. Pomagajo lahko tudi pri pripravi testov - poznavanje različnih stanj sistema lahko pomaga pri zagotavljanju popolnosti in pravilnosti testov.

17.2 Simulacija razvoja informacijskega sistema za bankomat

17.2.1 Predstavitev problema

Celovit postopek razvoja informacijske rešitve bomo izvedli s pomočjo **OO pristopa**, in sicer v naslednjih korakih:

- **zbiranje zahtev** (glej poglavje 17.2.2),
- **analiza** (identifikacija primerov uporabe, podrobnih tokov dogodkov, opredelitev funkcionalnih testov in analiza razredov) (glej poglavje 17.2.3),
- **načrt**
 - **pregledni načrt** (dodelitev odgovornosti razredom, razredni diagrami, diagrami stanj in sodelovanja) (glej poglavje 17.2.4) in
 - **podroben načrt** (atributi, metode, paketi) (glej poglavje 17.2.5) ter
- **implementacija** (Java) (glej poglavje 17.2.6).

17.2.2 Zajem zahtev

Razvita programska oprema bo skrbela za delovanje bankomata, ki je sestavljen iz naslednjih komponent:

- **bralca bančnih kartic**,

- **tipkovnice in zaslona** za komunikacijo s stranko,
- **predala za oddajo kuvert**,
- **reže za denar**,
- **tiskalnika za tiskanje potrdil** in
- **varnostnega stikala** za zagon oz. zaustavitev naprave s strani operaterja.

Bankomat lahko **hkrati** uporablja **ena stranka**. Stranka ob začetku interakcije **vstavi bančno kartico** in **vnese številko PIN**, kjer se oba podatka nato posredujeta banki v preverjanje. Stranka lahko nato **izvede eno ali več transakcij**. Kartica ostane v bankomatu, dokler stranka ne želi opraviti nobene transakcije več.

Bankomat mora stranki **nuditi** naslednje **storitve**:

- Stranka lahko **dvigne denar** iz poljubnega računa, ki je povezan z bančno kartico, v bankovcih vrednosti večkratnika 10 €.
- Stranka lahko **položi denar** na poljuben račun, ki je povezan z bančno kartico, glede na denar oz. čeke v kuverti. Stranka vnese višino depozita, ki se nato ročno preveri s strani operaterja bankomata. Potrditev razpoložljivosti računa za depozit mora biti s strani banke potrjena pred sprejetjem kuverte.
- Stranka lahko izvede **prenos med poljubnima računoma**, povezanima z vstavljenimi bančnimi karticami.
- Stranka lahko zahteva **izpis stanja na poljubnem računu**, povezanem z vstavljenimi bančnimi karticami.

Stranka lahko v vsakem trenutku prekine transakcijo s pritiskom na **gumb Prekliči**. Bankomat pri vsaki transakciji komunicira z banko in pridobi **potrditev izvedljivosti transakcije**. Transakcija je namreč zaključena, ko **jo potrdi banka**. V primeru depozita se posreduje banki še dodatno sporočilo, da je stranka na bankomatu oddala kuvert. Če stranka v predpisanem časovnem obdobju kuverte z denarjem ne odda ali pritisne tipko **Prekliči**, se dodatnega sporočila banki ne posreduje.

Če banka ugotovi, da je **PIN številka napačna**, jo mora stranka vnesti ponovno, da lahko uspešno izvede transakcijo. V primeru, da stranka **3x napačno vnese PIN**, bankomat **zadrži bančno kartico in stranka mora kontaktirati svojo banko**. Če transakcija ni uspešna zaradi razloga, ki ni napačna PIN številka, bo **bankomat obvestilo o napaki izpisal na zaslon in vprašal stranko, ali želi izvesti še kakšno transakcijo**.

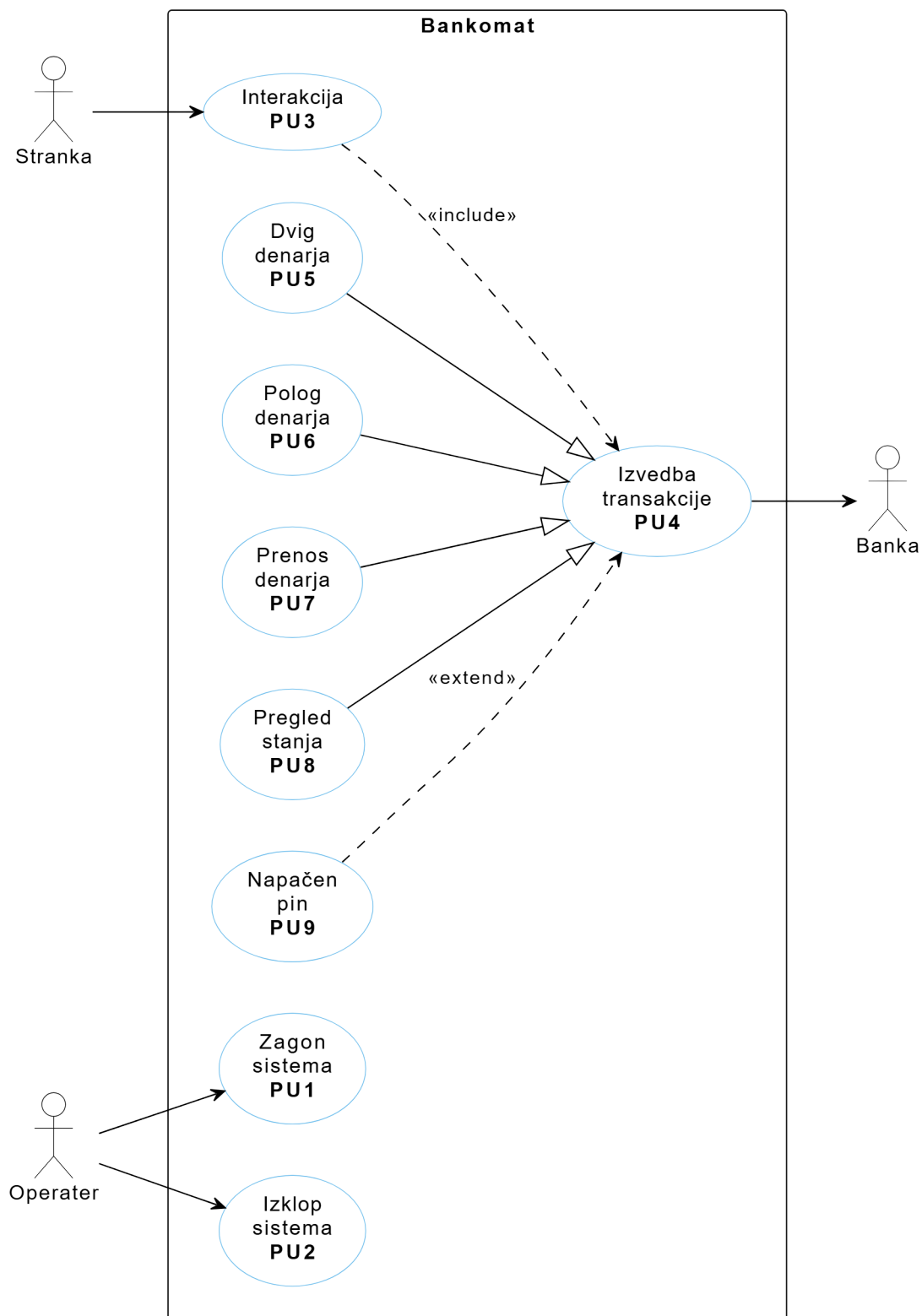
Bankomat stranki omogoči **izpis potrdila** o opravljeni transakciji, ki vsebuje datum, uro, lokacijo bankomata, tip transakcije, račun, znesek in končno stanje na računu.

Bankomat ima na voljo **varnostno stikalo za vklop oz. izklop**. Ko operater vklopi bankomat, mora preveriti točen znesek denarja, ki je na voljo. Bankomat se lahko izklopi le takrat, ko ga stranka ne uporablja. Ko je bankomat izklopljen, lahko operater odstrani kuverte z depoziti in ponovno naloži denar ter prazen papir za izpis potrdil.

Bankomat hrani tudi svoj **interni dnevnik transakcij** za lažje obvladovanje morebitnih napak, do katerih lahko pride zaradi strojne napake med transakcijo. V dnevnik se beležijo **vsi zagoni in ustavitve bankomata ter vsako poslano sporočilo banki (vključno z odgovorom)**. Dnevniški zapisi vsebujejo podatke o številki kartice in zneskih, vendar zaradi varnostnih razlogov **ni nikoli podatka o PIN številki**.

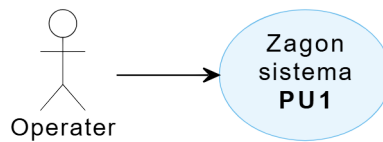
17.2.3 Analiza

17.2.3.1 Diagram primerov uporabe (DPU)



Slika 17.32: Diagram primerov uporabe (DPU)  PlantUML

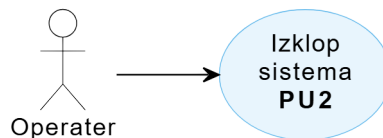
17.2.3.2 Opis PU1: Zagon sistema



Slika 17.33: **PU1**: Zagon sistema PlantUML

- Zagon sistema se izvede, ko operater vklopi stikalo.
- Operater pri tem vnese količino denarja, ki je trenutno v bankomatu, ter vzpostavi povezavo z banko.
- Zatem se nadaljuje ponujanje storitev strankam.

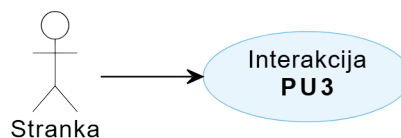
17.2.3.3 Opis PU2: Izklop sistema



Slika 17.34: **PU2**: Izklop sistema PlantUML

- Izklop sistema se opravi takrat, ko bankomata ne uporablja nobena stranka, in sicer operater izklopi stikalo.
- Povezava z banko se v tem trenutku prekine in operater lahko odstrani kuverte z depoziti, napolni denar in prazen papir za tiskanje potrdil.

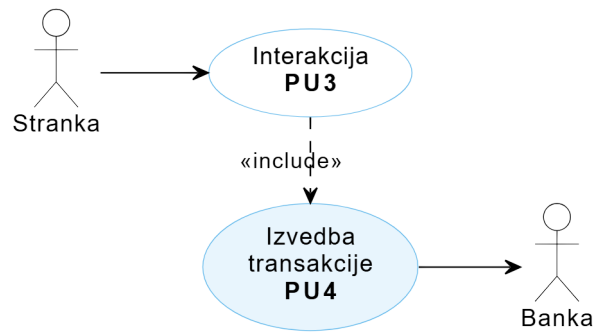
17.2.3.4 Opis PU3: Interakcija



Slika 17.35: **PU3**: Interakcija PlantUML

- Do interakcije pride, ko stranka vstavi kartico v bralnik kartic bankomata.
- Bankomat kartico povleče in jo prebere.
 - Če bralnik kartice ne more prebrati, jo izvrže in na zaslonu izpiše obvestilo o napaki.
- Bankomat od stranke zahteva vnos PIN številke in šele takrat ji je dovoljeno izvajati transakcije, ki jih izbere iz menija.
- Po vsaki transakciji stranka izbere, ali želi izvesti še kakšno transakcijo.
- Ko stranka zaključi s transakcijami, bankomat kartico izvrže in jo vrne stranki ter na ta način zaključi interakcijo.
- Če je transakcija prekinjena zaradi prevelikega števila napačno vnesenih PIN števil, se interakcija ravno tako zaključi, vendar kartica ostane v bankomatu.
- Stranka lahko med interakcijo le-to prekine s pritiskom na gumb Prekliči.

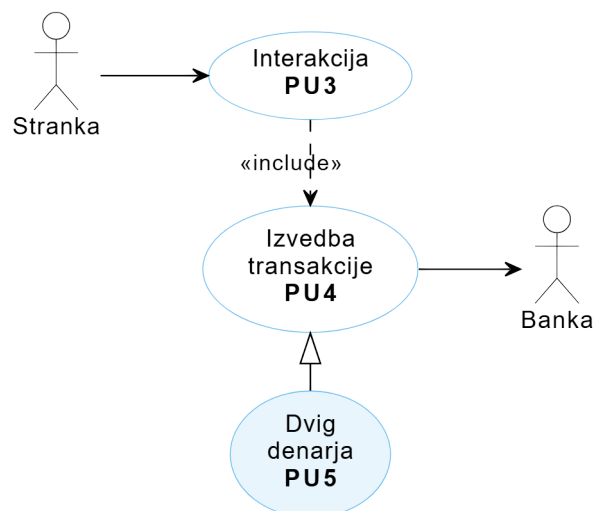
17.2.3.5 Opis PU4: Izvedba transakcije



Slika 17.36: **PU4**: Izvedba transakcije PlantUML

- Izvedba transakcije je **abstraktna generalizacija**. Vsak specifičen konkreten tip implementacije vsebuje svoje operacije. Tok dogodkov je podan na tem mestu in opisuje skupno delovanje.
- Izvedba transakcije se **požene znotraj interakcije**, ko stranka izbere tip transakcije iz menija. V nadaljevanju stranka poda ustrezne podrobnosti (npr. željen račun, znesek itd.). Transakcija je nato posredovana banki skupaj s potrebnimi informacijami o kartici in številki PIN.
- Če banka transakcijo potrdi, se izvedejo še ustrezne akcije (npr. oddaja denarja ali polog s položnico), nato pa se natisne še potrdilo o transakciji. Zatem se stranka odloči, ali želi izvesti še kakšno transakcijo.
- Če banka sporoči, da je številka PIN napačna, se izvede drug primer uporabe in se nato morebiti vrne izvajanje nazaj v ta primer uporabe. Če bankomat kartico zadrži zaradi prevelikega števila napačnih vnosov PIN številke, stranka ne bo mogla več izbirati akcij.
- Če stranka transakcijo prekliče ali če je neuspešna zaradi drugega razloga kot napačen PIN, se na zaslonu prikaže opis napake in stranki se ponudi, da izbere drugo transakcijo.
- Stranka lahko prekliče transakcijo s klikom na gumb Prekliči.
- Vsa sporočila med banko in bankomatom v obe smeri so shranjena v internem dnevniku bankomata.

17.2.3.6 Opis PU5: Dvig denarja



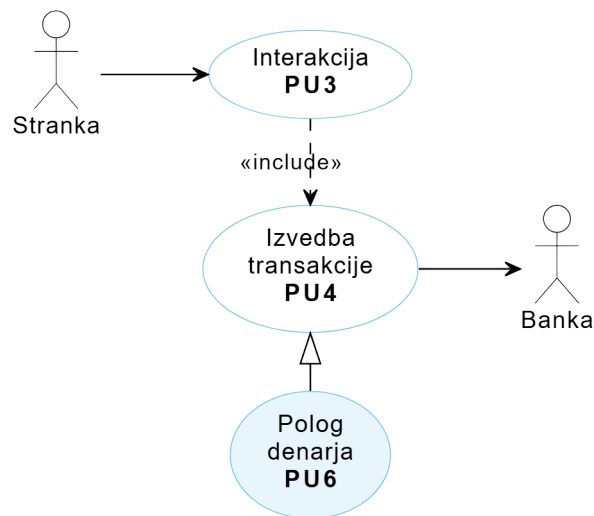
Slika 17.37: **PU5**: Dvig denarja PlantUML

- Pri dvigu denarja stranka izbere račun, iz katerega želi izvesti dvig in potem določi znesek iz menija možnih vrednosti. Sistem preveri, ali je v bankomatu dovolj denarja, preden pošlje transakcijo v obdelavo na banko. Če denarja ni dovolj, potem sistem stranko obvesti, naj izbere drug znesek. Če je

transakcija potrjena s strani banke, se stranki izda ustrezen znesek gotovine in natisne potrdilo. Izdaja denarja se shrani v interni dnevnik bankomata.

- Stranka lahko transakcijo prekliče s pritiskom na gumb Prekliči.

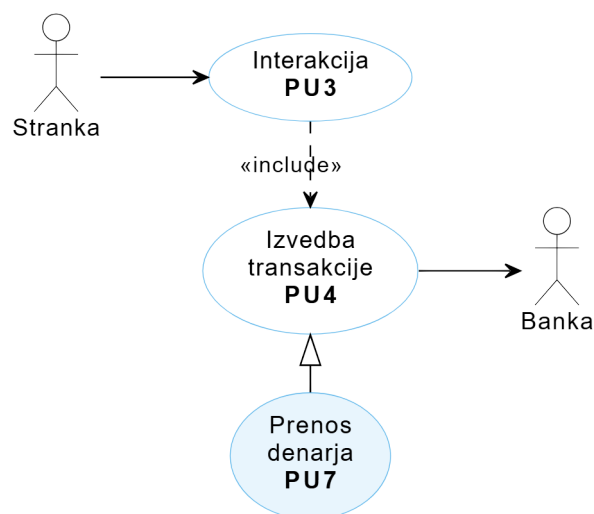
17.2.3.7 Opis PU6: Polog denarja



Slika 17.38: **PU6**: Polog denarja PlantUML

- Pri pologu denarja mora stranka izbrati račun, na katerega želi položiti denar, in višino depozita. V okviru transakcije se pri banki preveri, ali lahko bankomat opravi depozit na izbran račun. Če je transakcija potrjena, bankomat sprejme kuverto z denarjem ali čeki ter nato izda potrdilo. Ko je kuverta sprejeta, se posreduje še eno obvestilo banki, kjer se potrdi prenos denarja na račun, seveda po ročnem preverjanju vsebine kuverte. Omenjena komunikacija se hrani tudi v internem dnevniku bankomata.
- Depozit lahko stranka prekliče s pritiskom na gumb Prekliči vsak trenutek pred oddajo kuverte. Prav tako se transakcija prekliče, če stranka ne odda kuverte z denarjem v določenem časovnem obdobju.

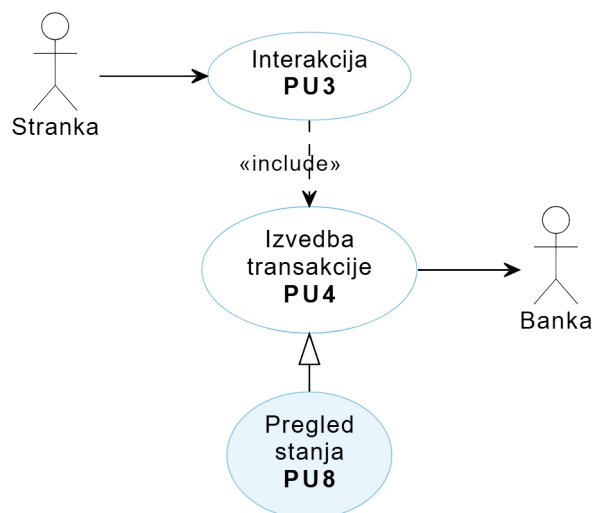
17.2.3.8 Opis PU7: Prenos denarja



Slika 17.39: **PU7**: Prenos denarja PlantUML

- Pri prenosu denarja mora stranka izmed vseh možnih računov izbrati tistega, iz katerega denar prenaša, in drug račun, na katerega denar prenaša, ter vsoto. Če banka potrdi transakcijo, bankomat natisne potrdilo. Omenjena komunikacija se hrani tudi v internem dnevniku bankomata.
- Prenos denarja lahko stranka prekine s pritiskom na tipko Prekliči.

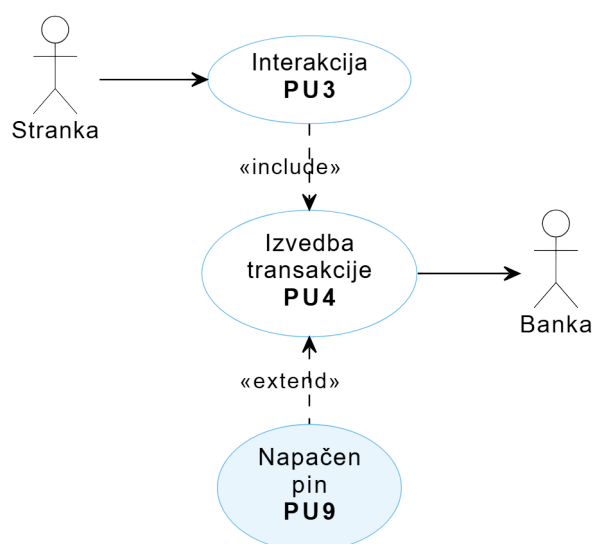
17.2.3.9 Opis PU8: Pregled stanja



Slika 17.40: **PU8**: Pregled stanja PlantUML

- Stranka iz menija izbere pregled stanja za posamezen račun. Ko banka vrne stanje, se izpiše potrdilo. Omenjena komunikacija se hrani tudi v internem dnevniku bankomata.
- Stranka lahko akcijo vedno prekliče s klikom na gumb Prekliči.

17.2.3.10 Opis PU9: Napačen PIN



Slika 17.41: **PU9**: Napačen PIN PlantUML

- PU se začne izvajati, ko pri poskusu izvedbe transakcije banka sporoči, da je stranka vnesla napačno PIN številko.
- Stranka mora nato ponovno vnesti PIN in prvotna transakcija se zopet posreduje banki v potrditev.
 - Če banka v tem trenutku transakcijo potrdi ali jo zavrne zaradi kakšnega drugega razloga, se osnovni PU nadaljuje, sicer se postopek ponovnega vnosa PIN številke ponovi.

- Ko je enkrat PIN številka pravilno zajeta, se uporablja za trenutno transakcijo in vse nadaljnje v okviru interakcije.
- Če stranka 3x napačno vnese PIN številko, bankomat obdrži kartico in na zaslonu se izpiše napaka ter priporočilo, naj stranka kontaktira svojo banko. S tem se zaključi tudi interakcija.
- Če stranka med tem procesom pritisne gumb Prekliči, se transakcija prekliče.

17.2.3.11 Začetni funkcionalni testi

Primer uporabe	Funkcija, ki se testira	Začetno stanje sistema	Vhod	Pričakovan rezultat
PU1: Zagon sistema	Sistem se zažene, ko vklopimo stikalo.	Sistem je izklopljen.	Vklop stikala.	Sistem zahteva nastavitve začetne vsote denarja.
PU1: Zagon sistema	Sistem sprejme začetno vsoto denarja.	Sistem zahteva začetno vsoto denarja.	Vnos pravilne vsote denarja.	Sistem je vklopljen.
PU1: Zagon sistema	Povezava do banke je vzpostavljena.	Sistem je bil ravnokar vklopljen.	Izvedi ustrezno poizvedovalno transakcijo.	Sistem izpiše uspešen status povezave do banke.
PU2: Izklop sistema	Sistem se izklopi ob izklopu stikala.	Sistem je vklopljen in stranka ga ne uporablja.	Izklop stikala.	Sistem je izklopljen.
PU2: Izklop sistema	Povezava do banke se ob izklopu prekine.	Sistem je bil ravnokar izklopljen.		Potrditev prekinitve povezave s strani banke.
PU3: Interakcija	Sistem prebere strankino bančno kartico.	Sistem je vklopljen in ga ne uporablja nobena stranka.	Vstavi ustrezno bančno kartico.	Kartica je sprejeta. Sistem zahteva PIN. Kartica je izvržena.
PU3: Interakcija	Sistem zavrne poškodovano kartico.	Sistem je vklopljen in ga ne uporablja nobena stranka.	Vstavi poškodovano kartico.	Sistem prikaže obvestilo o napaki. Sistem je pripravljen na novo interakcijo.
PU3: Interakcija	Sistem sprejme PIN številko.	Sistem zahteva vnos PIN številke.	Vnesi PIN.	Sistem prikaže meni z različnimi možnimi transakcijami.
PU3: Interakcija	Sistem stranki dovoli izvesti transakcijo.	Sistem prikaže meni z različnimi možnimi transakcijami.	Izvedi transakcijo.	Sistem stranko vpraša, ali želi izvesti še kakšno transakcijo.
PU3: Interakcija	Sistem dovoli izvesti več transakcij v okviru ene interakcije.	Sistem stranko vpraša, ali želi izvesti še kakšno transakcijo.	Stranka potrdi.	Sistem prikaže meni z različnimi možnimi transakcijami.
PU3: Interakcija	Interakcija se konča, ko stranka ne želi opraviti nobene transakcije več.	Sistem stranko vpraša ali želi izvesti še kakšno transakcijo.	Stranka zavrne.	Sistem vrne kartico in je pripravljen na novo interakcijo.
PU4: Izvedba transakcije	Sistem ustrezno obravnava vnos napačne PIN številke.	Vstavljena je ustrezna bančna kartica.	Vnesi napačno PIN številko in poskusi izvesti transakcijo.	Izvede se "PU9: Napačen PIN".
PU5: Dvig denarja	Sistem zahteva od stranke izbiro računa.	Prikazan je meni s seznamom možnih transakcij.	Izberi "Dvig denarja".	Sistem prikaže meni s seznamom možnih transakcij.

Primer uporabe	Funkcija, ki se testira	Začetno stanje sistema	Vhod	Pričakovan rezultat
PU5: Dvig denarja	Sistem zahteva od stranke vnos zneska.	Prikazan je meni s seznamom računov.	Izberi transakcijski račun.	Sistem prikaže meni možnih zneskov dviga.
PU5: Dvig denarja	Sistem uspešno izvede zahtevano transakcijo, ki je izvedljiva.	Sistem prikazuje meni možnih zneskov dviga.	Izberi znesek, ki je na voljo v bankomatu in ni večji od stanja na računu.	Sistem izstavi denar. Sistem natisne potrdilo o uspešni transakciji in s trenutnim stanjem. Sistem zapiše akcijo (sporočilo banki in potrditev) v svoj interni dnevnik.
PU5: Dvig denarja	Sistem preveri, ali je dovolj denarja za izpolnitev zahteve.	Sistem je bil vklopljen z manjšim zneskom denarja, kot je bil zahtevan pri dvigu. Stranka zahteva dvig denarja.	Izberi znesek, ki je večji, kot je na voljo v bankomatu.	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro nižjega zneska.
PU5: Dvig denarja	Sistem preveri, ali je stanje na strankinem računu zadostno za izpolnitev zahteve.	Stranka zahteva dvig denarja.	Izberi znesek, ki je trenutno na voljo v bankomatu in je višji od stanja na strankinem računu.	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU5: Dvig denarja	Transakcija je lahko preklicana s strani stranke pred izbiro zneska.	Sistem prikazuje meni s seznamom računov.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU6: Polog denarja	Sistem zahteva od stranke izbiro računa za polog.	Prikazan je meni s seznamom možnih transakcij.	Izberi "Polog denarja".	Sistem prikaže meni možnih transakcij.
PU6: Polog denarja	Sistem zahteva od stranke vnos zneska.	Prikazan je meni s seznamom računov.	Izberi transakcijski račun.	Sistem zahteva od stranke vnos zneska.
PU6: Polog denarja	Sistem zahteva od stranke vstavitvev kuverte z denarjem.	Sistem zahteva od stranke vnos zneska.	Vnesi ustrezen znesek.	Sistem zahteva od stranke vstavitvev kuverte z denarjem.
PU6: Polog denarja	Sistem uspešno izvede zahtevano transakcijo, ki je izvedljiva.	Sistem zahteva od stranke vstavitvev kuverte z denarjem.	Vstavi kuverto z denarjem.	Sistem sprejme kuverto z denarjem. Sistem natisne potrdilo o uspešni transakciji s posodobljenim stanjem. Sistem zapiše akcijo (sporočilo banki, potrditev in prejetje kuverte) v svoj interni dnevnik.

Primer uporabe	Funkcija, ki se testira	Začetno stanje sistema	Vhod	Pričakovan rezultat
PU6: Polog denarja	Transakcija je lahko preklicana s strani stranke pred oddajo kuverte z denarjem.	Prikazan je meni s seznamom računov.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU6: Polog denarja	Transakcija je lahko preklicana s strani stranke pred oddajo kuverte z denarjem.	Sistem zahteva od stranke vnos zneska.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU6: Polog denarja	Transakcija je lahko preklicana s strani stranke pred oddajo kuverte z denarjem.	Sistem zahteva od stranke vstavitvev kuverte z denarjem.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU6: Polog denarja	Transakcija je samodejno preklicana, če se kuverta ne vstavi pravočasno.	Sistem zahteva od stranke vstavitvev kuverte z denarjem.	Počakaj, da zahteva časovno poteče.	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU7: Prenos denarja	Sistem zahteva od stranke izbiro računa, iz katerega se denar prenaša.	Prikazan je meni s seznamom možnih transakcij.	Izberi "Prenos denarja".	Sistem prikaže meni računov, iz katerih lahko prenesemo denar.
PU7: Prenos denarja	Sistem zahteva od stranke izbiro računa, na katerega se denar prenaša.	Prikazan je meni s seznamom računov.	Izberi transakcijski račun.	Sistem prikaže meni računov, na katere lahko prenesemo denar.
PU7: Prenos denarja	Sistem zahteva od stranke vnos zneska.	Prikazan je meni s seznamom računov.	Izberi varčevalni račun.	Sistem zahteva od stranke vnos zneska.
PU7: Prenos denarja	Sistem uspešno izvede zahtevano transakcijo, ki je izvedljiva.	Prikazana je zahteva za vnos zneska.	Vnesi veljaven znesek.	Sistem natisne potrdilo o uspešni transakciji s posodobljenim stanjem. Sistem zapiše akcijo (sporočilo banki in potrditev) v svoj interni dnevnik.
PU7: Prenos denarja	Transakcija je lahko preklicana s strani stranke pred vnosom zneska.	Prikazan je meni s seznamom računov, iz katerih se denar prenaša.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU7: Prenos denarja	Transakcija je lahko preklicana s strani stranke pred vnosom zneska.	Prikazan je meni s seznamom računov, na katere se denar prenaša.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.

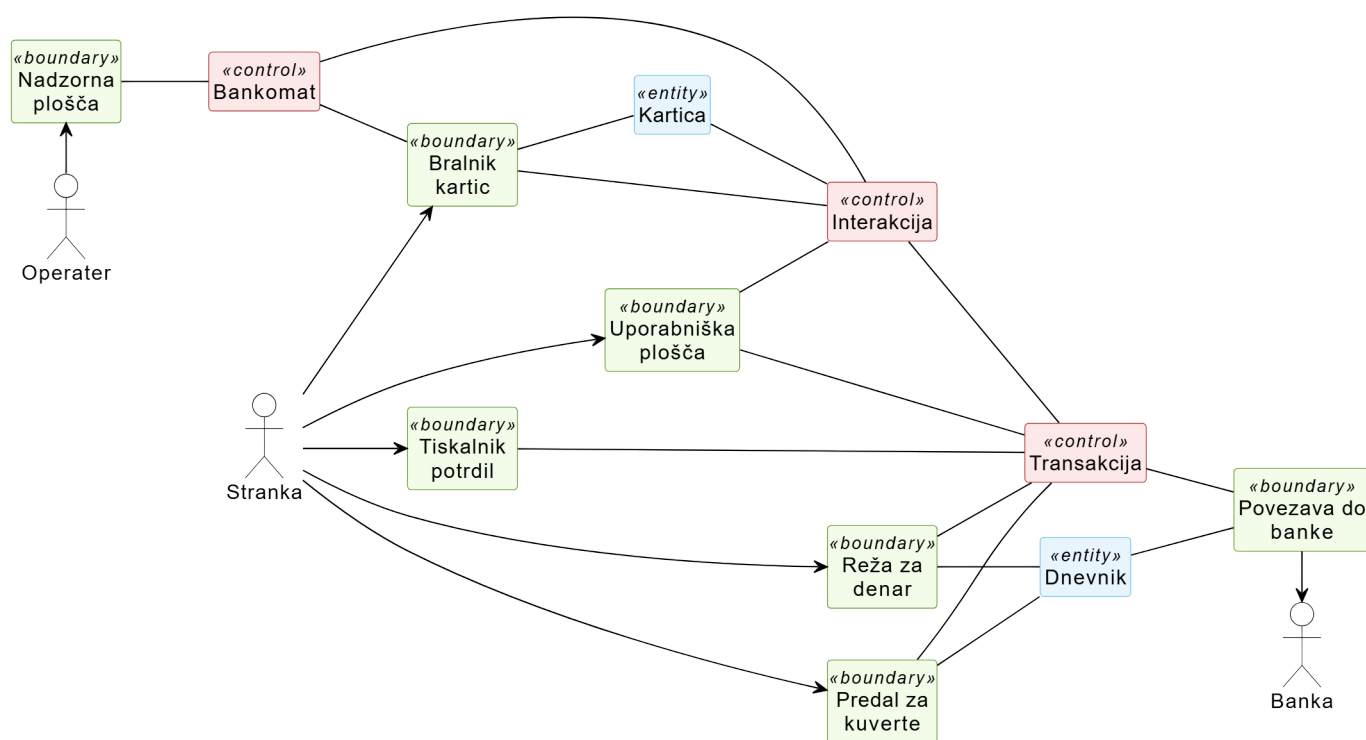
Primer uporabe	Funkcija, ki se testira	Začetno stanje sistema	Vhod	Pričakovan rezultat
PU7: Prenos denarja	Transakcija je lahko preklicana s strani stranke pred vnosom zneska.	Prikazana je zahteva za vnos zneska.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU8: Pregled stanja	Sistem zahteva od stranke izbiro računa za pregled stanja.	Prikazan je meni s seznamom možnih transakcij.	Izberi "Pregled stanja".	Sistem prikaže meni s seznamom računov.
PU8: Pregled stanja	Sistem uspešno izvede zahtevano transakcijo, ki je izvedljiva.	Prikazan je meni s seznamom računov.	Izberi transakcijski račun.	Sistem natisne potrdilo s pravilnim stanjem. Sistem zapiše akcijo (sporočilo banki in potrditev) v svoj interni dnevnik.
PU8: Pregled stanja	Transakcija je lahko preklicana s strani stranke pred izbiro računa.	Prikazan je meni s seznamom računov.	Pritisni tipko "Prekliči".	Sistem prikaže ustrezno obvestilo in ponudi stranki izbiro druge transakcije ali zaključek interakcije.
PU9: Napačen PIN	Stranka mora ponovno vnesti PIN številko.		Vnesi napačno PIN številko. Poskusi izvesti pregled stanja na strankinem transakcijskem računu.	Stranka mora ponovno vnesti PIN številko.
PU9: Napačen PIN	Ponovni vnos PIN številke je pravilen.	Prikazana je zahteva za ponoven vnos PIN številke.	Vnesi pravilno PIN številko.	Prvotna transakcija se uspešno izvede.
PU9: Napačen PIN	Pravilna PIN številka se uporabi za nadaljnje transakcije.	Napačna PIN številka je bila ponovno pravilno vnesena in prvotna transakcija je uspešno izvedena.	Izvedi še eno transakcijo.	Tudi dodatna transakcija se uspešno izvede.
PU9: Napačen PIN	Ponovni vnos PIN številke je napačen.	Prikazana je zahteva za ponoven vnos PIN številke.	Vnesi napačno PIN številko.	Prikazano je sporočilo o napaki in ponovno se zahteva vnos PIN številke.
PU9: Napačen PIN	Drugi ponovni vnos PIN številke je pravilen.	Prikazana je zahteva za ponoven vnos PIN številke.	Prvič vnesi napačno PIN številko, drugič pa pravilno.	Prvotna transakcija se uspešno izvede.
PU9: Napačen PIN	Tretji ponovni vnos PIN številke je pravilen.	Prikazana je zahteva za ponovni vnos PIN številke.	Prvič in drugič vnesi napačno PIN številko, tretjič pa pravilno.	Prvotna transakcija se uspešno izvede.

Primer uporabe	Funkcija, ki se testira	Začetno stanje sistema	Vhod	Pričakovan rezultat
PU9: Napačen PIN	Trije napačni vnosi PIN številke, odvzem kartice in prekinitev interakcije.	Prikazana je zahteva za ponoven vnos PIN številke.	3x vnosi napačno PIN številko.	Prikazano je sporočilo o napaki. Bankomat zadrži kartico. Interakcija je prekinjena.

17.2.3.12 Razredi analize

Glede na zajem zahtev lahko identificiramo naslednje skupine razredov (glej sliko 17.42):

- **mejni razredi** (*angl. boundary*), ki predstavljajo komunikacijo z okolico:
 - nadzorna plošča, uporabniška plošča, bralnik kartic, tiskalnik potrdil, reža za denar, predal za kuverte, povezava do banke,
- **kontrolni razredi** (*angl. control*), ki vsebujejo poslovno logiko:
 - bankomat, interakcija, transakcija,
- **poslovni razredi** (*angl. entity*), ki predstavljajo podatke, s katerimi operiramo:
 - kartica, dnevnik.



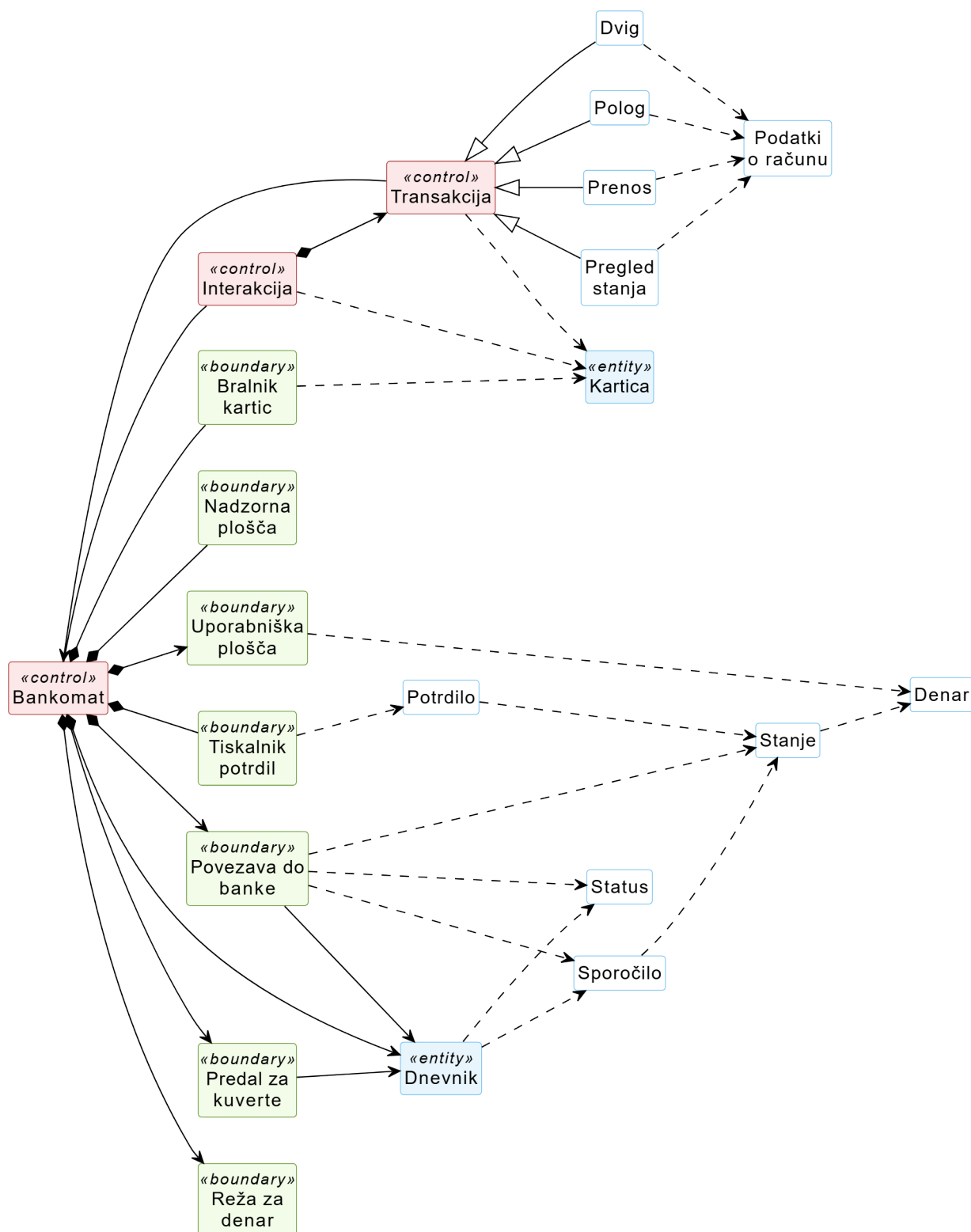
Slika 17.42: VOPC diagram PlantUML

17.2.4 Pregledni načrt

17.2.4.1 Razredni diagram

Pri določitvi osnovne strukture **razrednega diagrama** (glej sliko 17.43) do seznama razredov pridemo:

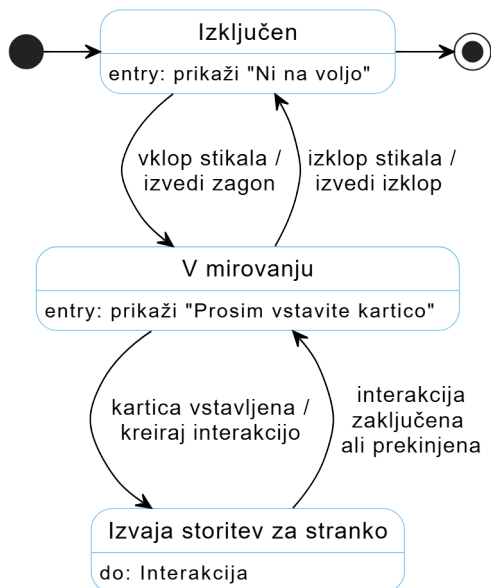
- med analizo pri **razredih analize**,
- pri **pregledu odgovornosti in povezav** med razredi CRC (angl. *W Class, Responsibility, Collaborator*),
 - **sporočilo** (posredovano banki),
 - **potrdilo** (natisnjeno stranki),
 - **status** (odgovor banke),
 - **stanje** (račun stranke na banki),
- pri izdelavi **podrobnega načrta ali kodiranja**,
 - **denar** (pri računih) in
 - **podatki o računu** (tipi računov ipd.).



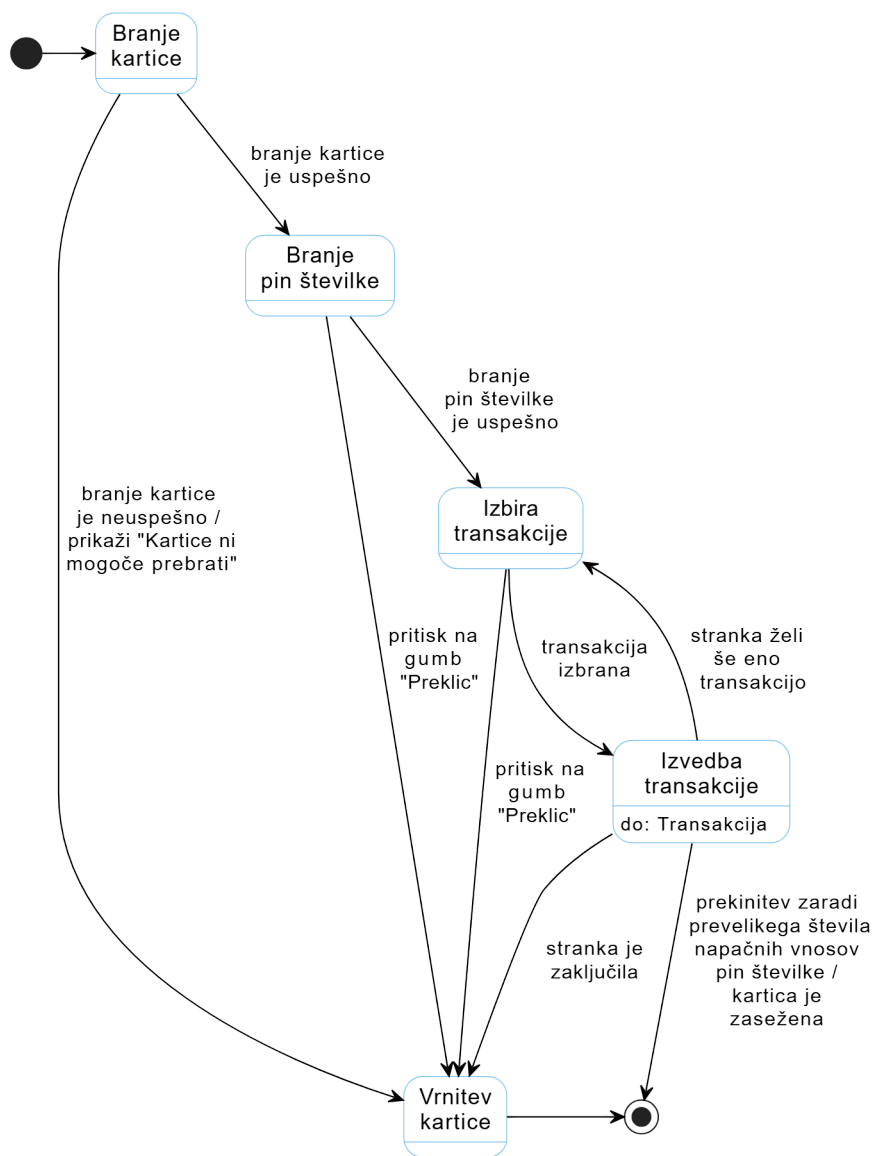
17.2.4.2 Diagram stanj

Pri pregledu objektov smo ugotovili, da obstajajo trije, ki so dovolj kompleksni in jih je potrebno predstaviti z diagramom stanj:

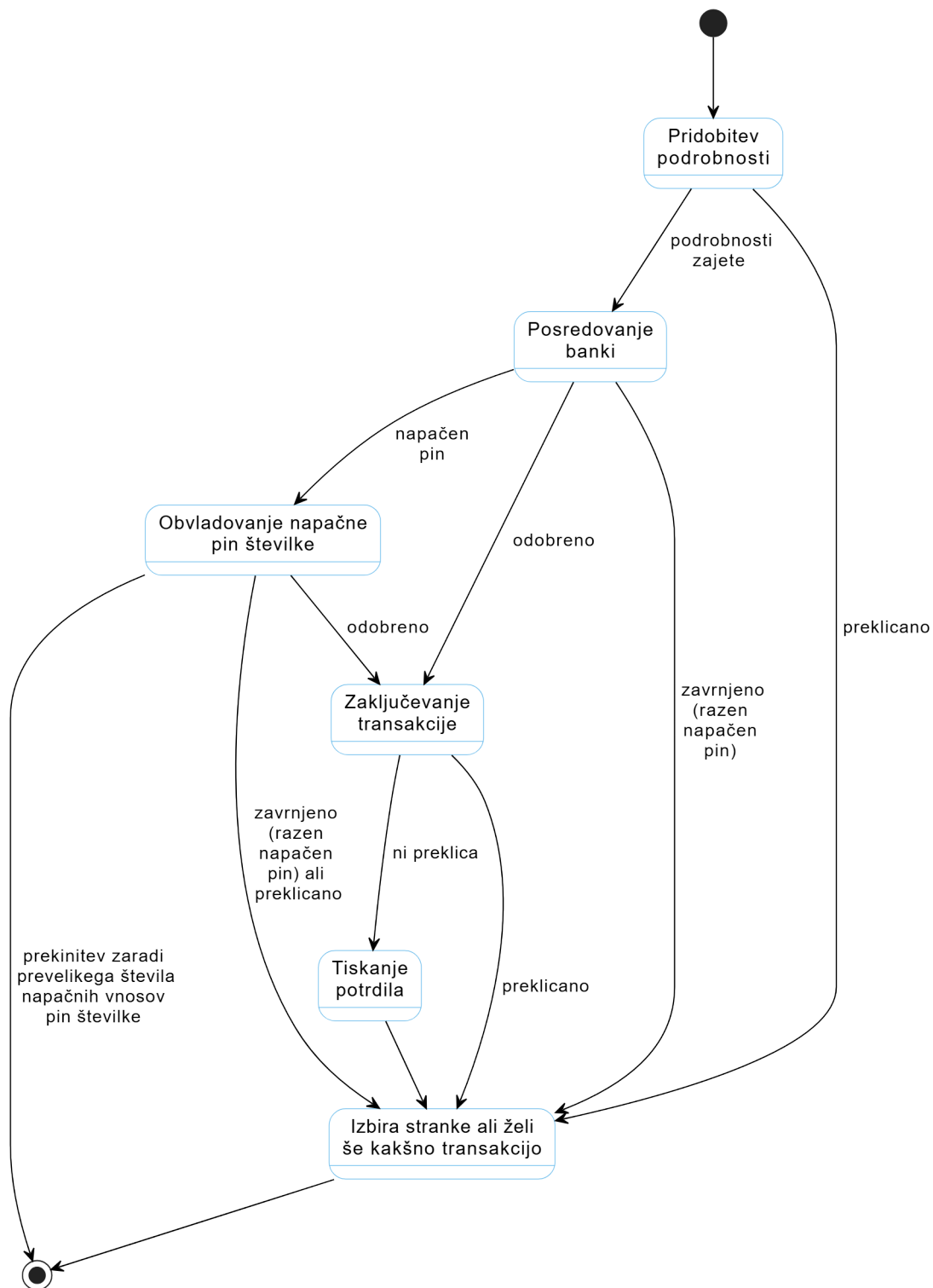
- objekt **bankomat** (glej diagram stanj 17.44),
- objekt **interakcija** (glej diagram stanj 17.45) in
- objekt **transakcija** (glej diagram stanj 17.46).



Slika 17.44: Diagram stanj za objekt **bankomat**  PlantUML



Slika 17.45: Diagram stanj za **objekt interakcija** PlantUML

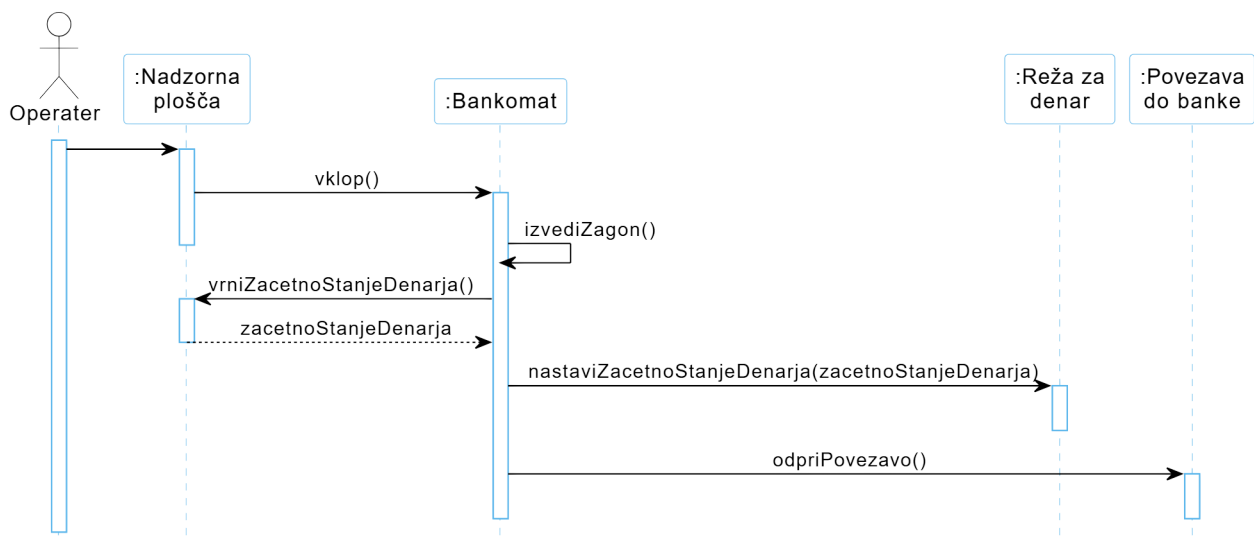


Slika 17.46: Diagram stanj za **objekt transakcija** PlantUML

17.2.4.3 Diagram interakcije

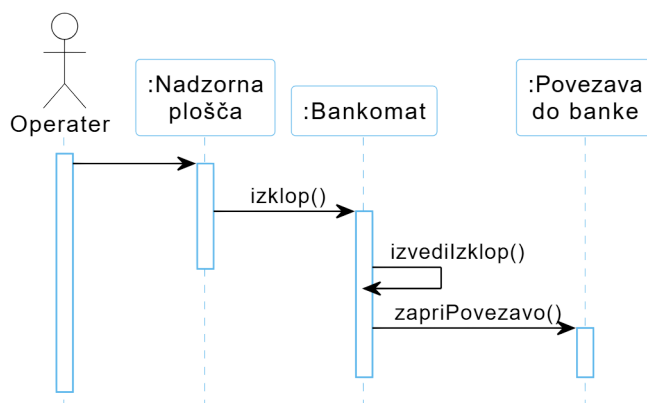
Za vsakega izmed primerov uporabe (na podlagi opisov iz analize v poglavju 17.2.3) pripravimo **diagram interakcije**, vsaj za osnovni tok (lahko tudi za alternativne tokove).

17.2.4.3.1 PU1: Zagon sistema



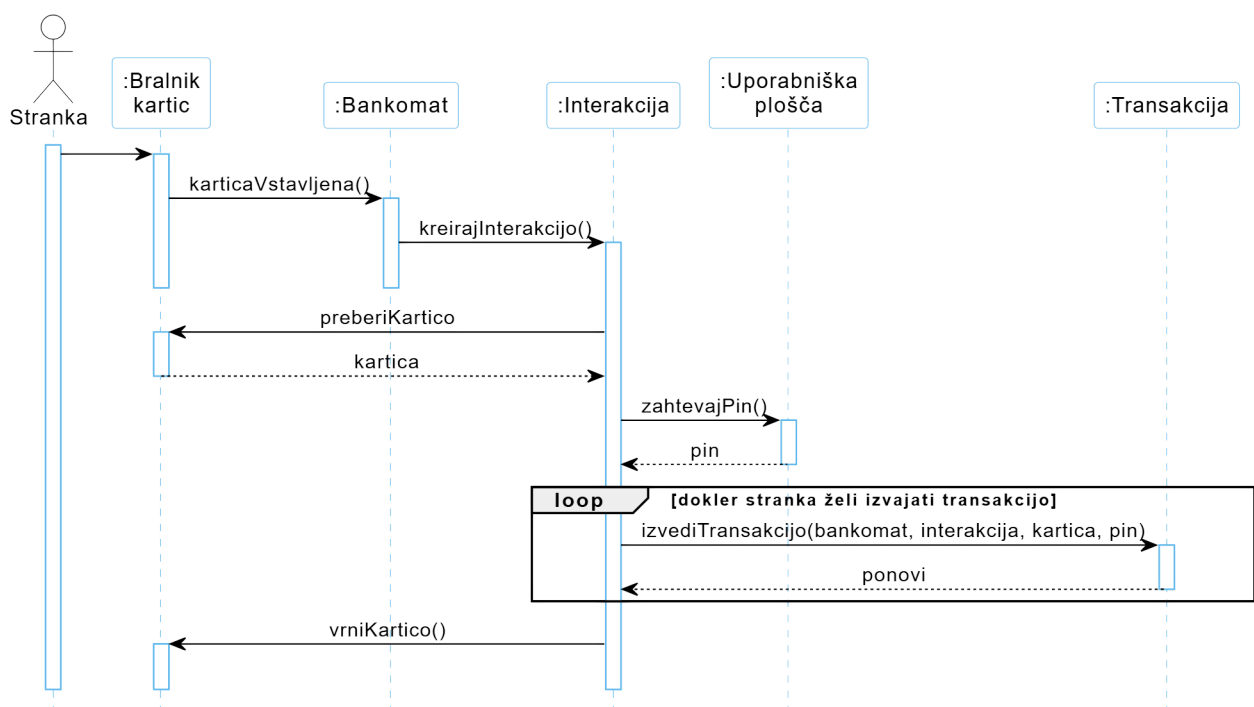
Slika 17.47: Diagram zaporedja za **PU1**: Zagon sistema PlantUML

17.2.4.3.2 PU2: Izklop sistema



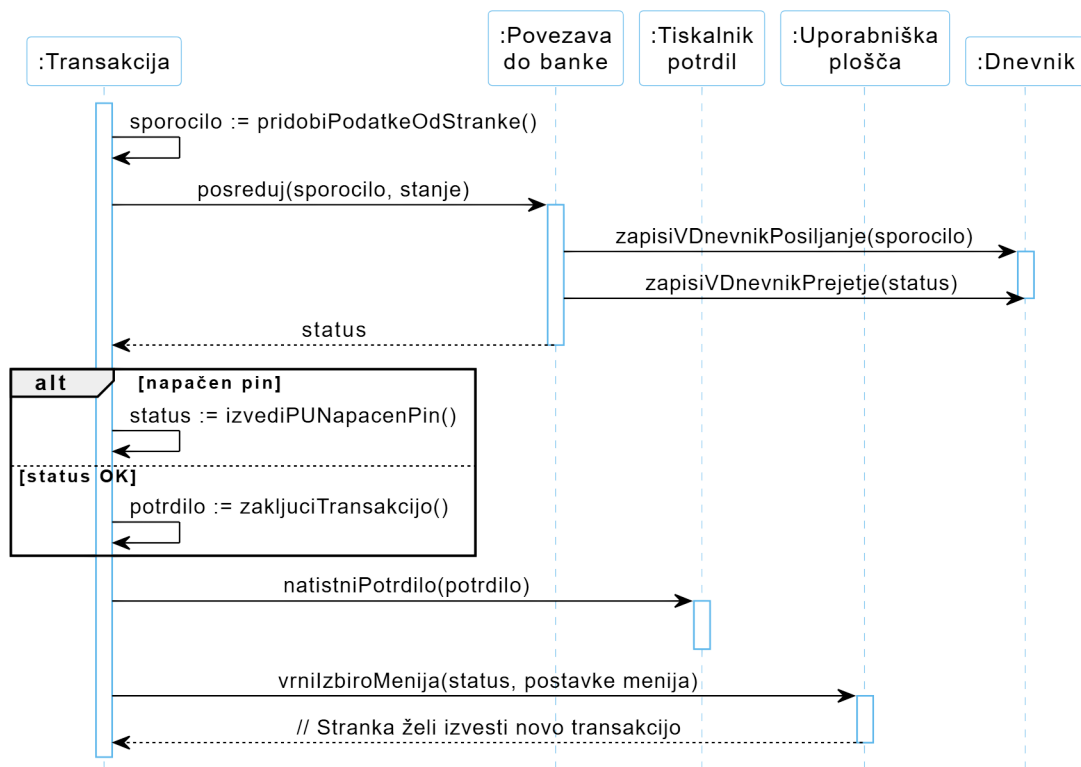
Slika 17.48: Diagram zaporedja za **PU2**: Izklop sistema PlantUML

17.2.4.3.3 PU3: Interakcija



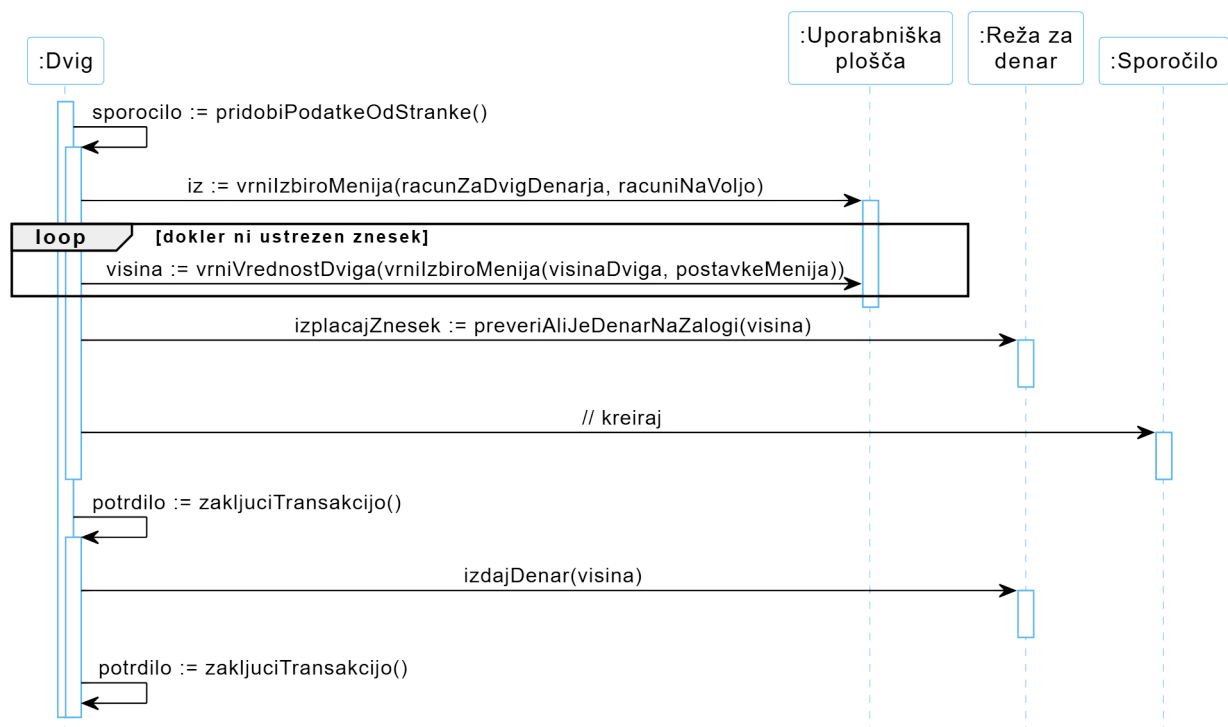
Slika 17.49: Diagram zaporedja za **PU3**: Interakcija PlantUML

17.2.4.3.4 PU4: Izvedba transakcije



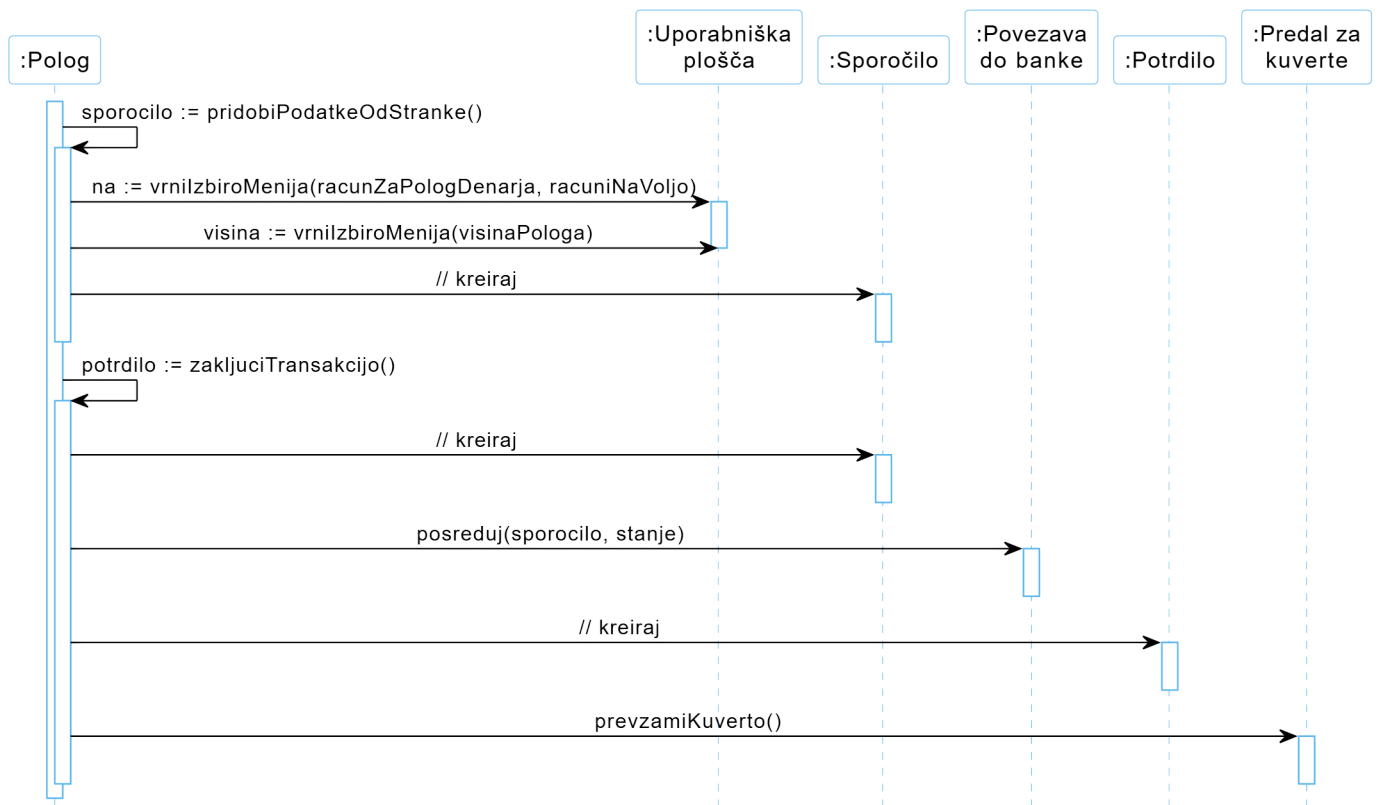
Slika 17.50: Diagram zaporedja za **PU4**: Izvedba transakcije PlantUML

17.2.4.3.5 PU5: Dvig denarja



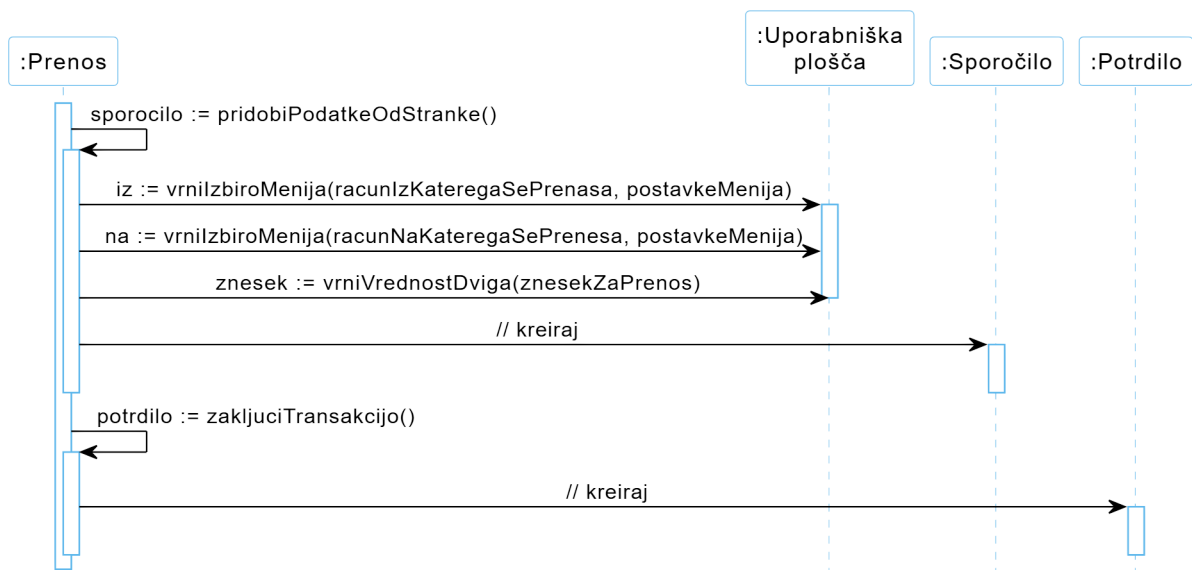
Slika 17.51: Diagram zaporedja za **PU5**: Dvig denarja PlantUML

17.2.4.3.6 PU6: Polog denarja



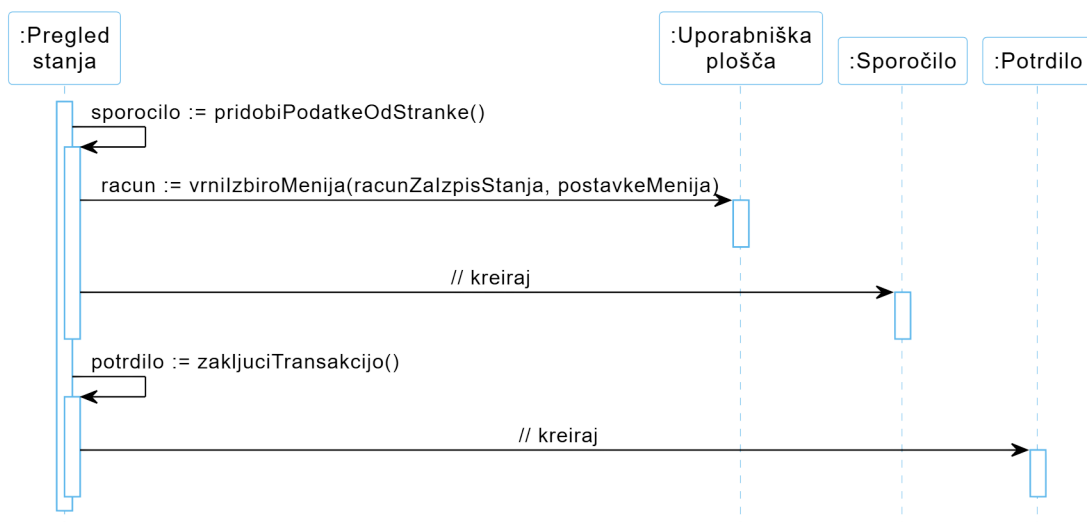
Slika 17.52: Diagram zaporedja za **PU6**: Polog denarja PlantUML

17.2.4.3.7 PU7: Prenos denarja



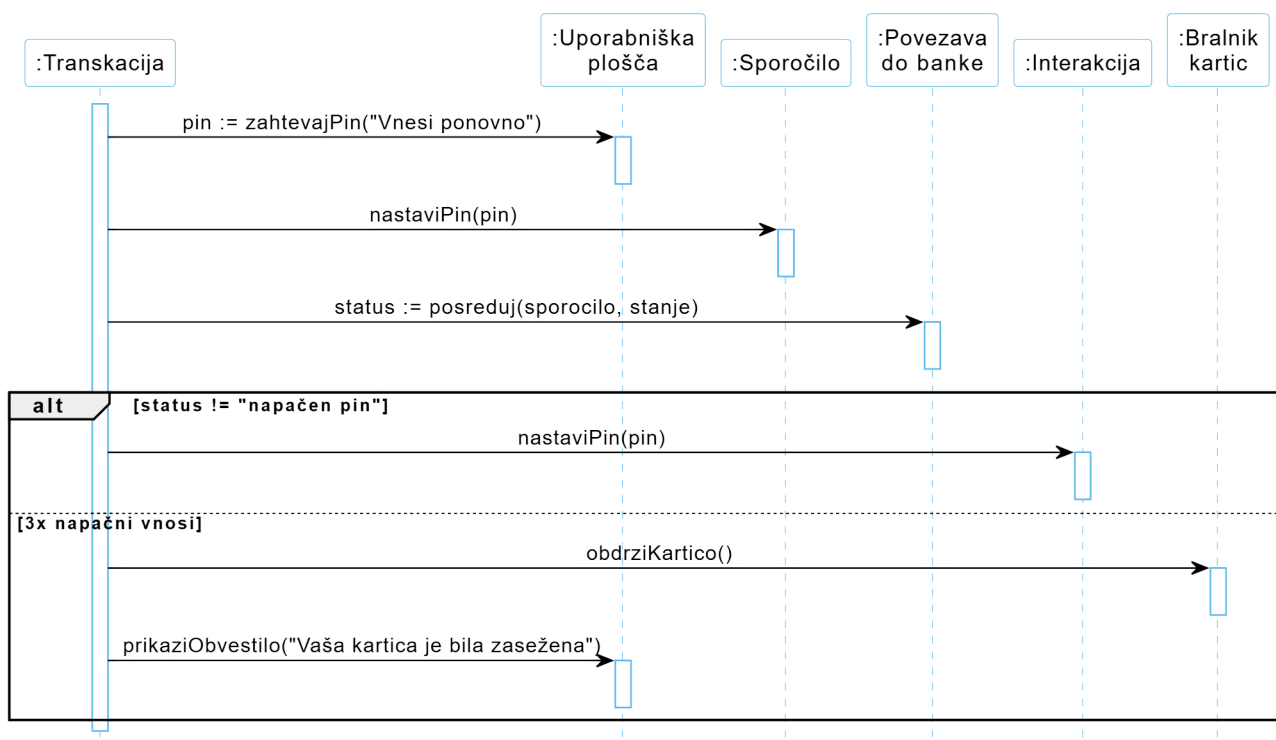
Slika 17.53: Diagram zaporedja za **PU7**: Prenos denarja PlantUML

17.2.4.3.8 PU8: Pregled stanja



Slika 17.54: Diagram zaporedja za **PU8**: Pregled stanja PlantUML

17.2.4.3.9 PU9: Napačen PIN



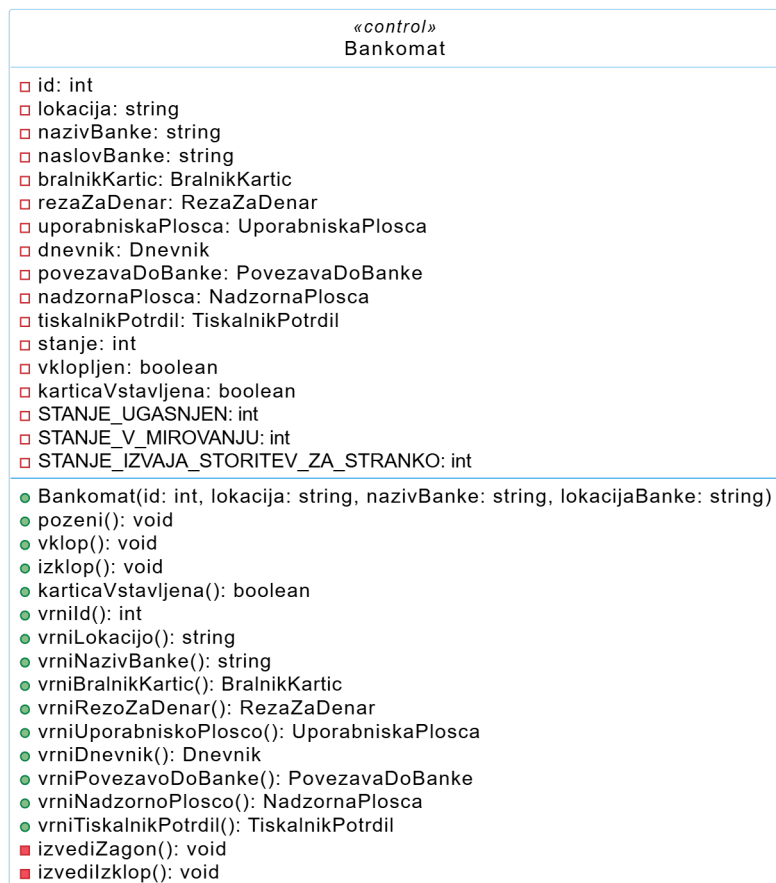
Slika 17.55: Diagram zaporedja za **PU9**: Napačen pin PlantUML

17.2.5 Podroben načrt

Ključna naloga podrobnega načrta je **podrobno opredeliti attribute in metode** vsakega razreda.

V nadaljevanju bomo podali podrobnosti zgolj nekaj ključnih razredov:

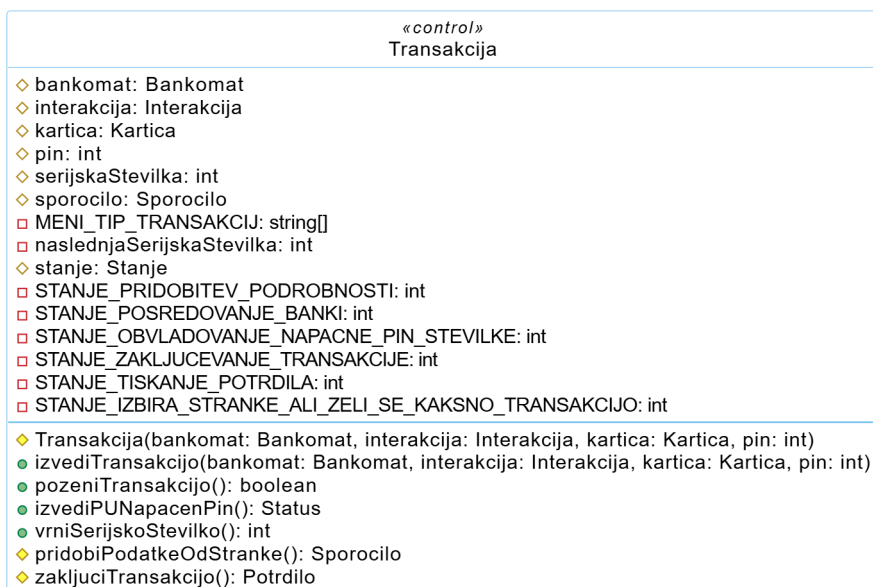
- **bankomat** (glej sliko 17.56),
- **interakcija** (glej sliko 17.57),
- **transakcija** (glej sliko 17.58),
- **bralnik kartic** (glej sliko 17.59) in
- **status** (glej sliko 17.60).



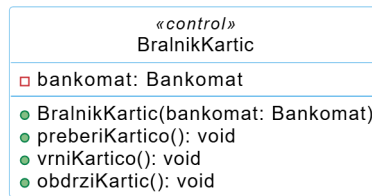
Slika 17.56: Razred **bankomat**  PlantUML



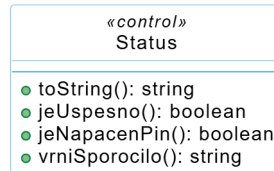
Slika 17.57: Razred **interakcija**  PlantUML



Slika 17.58: Razred **transakcija**  PlantUML

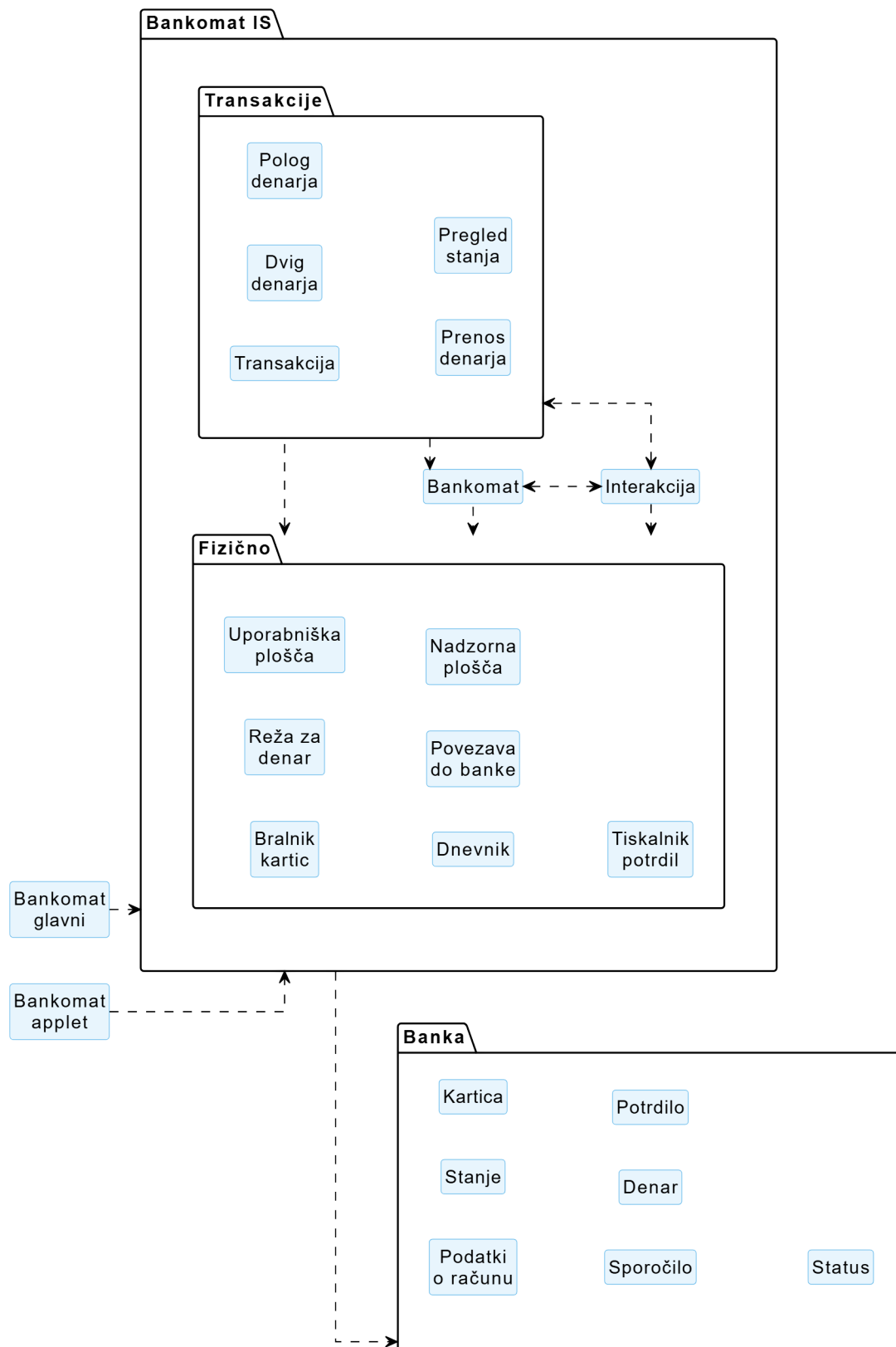


Slika 17.59: Razred **bralnik kartic**  PlantUML



Slika 17.60: Razred **status**  PlantUML

17.2.5.1 Paketni diagram



Slika 17.61: Paketni diagram PlantUML

17.2.6 Implementacija

BankomatApplet.java

```

import java.awt.*;
import java.applet.Applet;
import bankomat.Bankomat;
import simulacija.Simulacija;

public class BankomatApplet extends Applet {
    public void init () {

        Bankomat bankomat = new Bankomat (42, "Nama", "NLB",
            null /* Pri simulaciji ne bomo dejansko komunicirali z banko */);
        Simulacija simulacija = new Simulacija (bankomat);

        new Thread (bankomat).start ();

        Panel gui = simulacija.vrniGUI ();
        setBackground(gui.vrniOzadje ());
        add (gui);
    }
}

```

Bankomat.java

«control» Bankomat
□ id: int □ lokacija: string □ nazivBanke: string □ naslovBanke: string □ bralnikKartic: BralnikKartic □ rezaZaDenar: RezaZaDenar □ uporabniskaPlosca: UporabniskaPlosca □ dnevnik: Dnevnik □ povezavaDoBanke: PovezavaDoBanke □ nadzornaPlosca: NadzornaPlosca □ tiskalnikPotrdil: TiskalnikPotrdil □ stanje: int □ vklopljen: boolean □ karticaVstavljena: boolean □ STANJE_UGASNJEN: int □ STANJE_V_MIROVANJU: int □ STANJE_IZVAJA_STORITEV_ZA_STRANKO: int
● Bankomat(id: int, lokacija: string, nazivBanke: string, lokacijaBanke: string) ● pozeni(): void ● vklop(): void ● izklop(): void ● karticaVstavljena(): boolean ● vrniId(): int ● vrniLokacijo(): string ● vrniNazivBanke(): string ● vrniBralnikKartic(): BralnikKartic ● vrniRezoZaDenar(): RezaZaDenar ● vrniUporabniskoPlosco(): UporabniskaPlosca ● vrniDnevnik(): Dnevnik ● vrniPovezavoDoBanke(): PovezavaDoBanke ● vrniNadzornoPlosco(): NadzornaPlosca ● vrniTiskalnikPotrdil(): TiskalnikPotrdil ■ izvediZagon(): void ■ izvediIzklop(): void

```

package bankomatIs;

import java.net.String;
import bankomatIs.fizicno.*;
import banka.Kartica;
import banka.Denar;

public class Bankomat implements Runnable {

    public Bankomat (int id, String lokacija, String nazivBanke, String naslovBanke) {
        this.id = id;
        this.lokacija = lokacija;
        this.nazivBanke = nazivBanke;
        this.naslovBanke = naslovBanke;

        dnevnik = new Dnevnik();
        bralnikKartic = new BralnikKartic(this);
        rezaZaDenar = new RezaZaDenar(dnevnik);
        uporabniskaPlosca = new UporabniskaPlosca();
        povezavaDoBanke = new PovezavaDoBanke(dnevnik, naslovBanke);
        nadzornaPlosca = new NadzornaPlosca(this);
        tiskalnikPotrdil = new TiskalnikPotrdil();

        stanje = STANJE_UGASNJEN;
        vklopljen = false;
        karticaVstavljena = false;
    }

    public void run() {
        Interakcija trenutnaInterakcija = null;

        while (true) {
            switch (stanje) {
                case STANJE_UGASNJEN:
                    uporabniskaPlosca.prikaziObvestilo ("Trenutno ni na voljo");
                    synchronized (this) {
                        try {
                            wait();
                        } catch (InterruptedException e) { }
                    }
                    if (vklopljen) {
                        izvediZagon ();
                        stanje = STANJE_V_MIROVANJU;
                    }
                }
            }
        }
    }
}

```



```

    }

    break;
case STANJE_V_MIROVANJU:
    uporabniskaPlosa.prikaziObvestilo ("Prosim vstavite kartico");
    karticaVstavljena = false;
    synchronized (this) {
        try {
            wait();
        } catch (InterruptedException e) { }
    }
    if (karticaVstavljena) {
        trenutnaInterakcija = new Interakcija (this);
        stanje = STANJE_IZVAJA_STORITEV_ZA_STRANKO;
    } else if (! vklopljen) {
        izvediIzklop();
        stanje = STANJE_UGASNJEN;
    }
    break;
case STANJE_IZVAJA_STORITEV_ZA_STRANKO:
    trenutnaInterakcija.kreirajInterakcijo ();
    status = STANJE_V_MIROVANJU;
    break;
}
}
}

public synchronized void vklop () {
    vklopljen = true;
    notify();
}

public synchronized void izklop () {
    vklopljen = false;
    notify();
}

public synchronized void karticaVstavljena () {
    karticaVstavljena = true;
    notify ();
}

public int vrniID () {
    return id;
}

```

```
public String vrniLokacijo () {
    return lokacija;
}

public String vrniNazivBanke () {
    return nazivBanke;
}

public BralnikKartic vrniBralnikKartic () {
    return bralnikKartic;
}

public RezaZaDenar vrniRezoZaDenar () {
    return rezaZaDenar;
}

public UporabniskaPlosca vrniUporabniskoPlosco () {
    return uporabniskaPlosca;
}

public Dnevnik vrniDnevnik () {
    return dnevnik;
}

public PovezavaDoBanke vrniPovezavoDoBanke () {
    return povezavaDoBanke;
}

public NadzornaPlosca vrniNadzornoPlosco () {
    return nadzornaPlosca;
}

public TiskalnikPotrdil vrniTiskalnikPotrdil () {
    return tiskalnikPotrdil;
}

private void IzvediZagon () {
    Denar zacetniZnesek = nadzornaPlosca.vrniZacetnoStanjeDenarja ();
    rezaZaDenar.nastaviZacetnoStanjeDenarja (zacetniZnesek);
    povezavaDoBanke.odpriPovezavo ();
}

private void izvediIzklop () {
```

```

    povezavaDoBanke.zapriPovezavo ();
}

private int id;
private String lokacija;
private String nazivBanke;
private String naslovBanke;
private BralnikKartice bralnikKartice;
private RezaZaDenar rezaZaDenar;
private UporabniskaPlosca uporabniskaPlosca;
private Dnevnik dnevnik;
private PovezavaDoBanke povezavaDoBanke;
private NadzornaPlosca nadzornaPlosca;
private TiskalnikPotrdil tiskalnikPotrdil;
private int stanje;
private boolean vklopljen;
private boolean karticaVstavljena;
private static final int STANJE_UGASNJEN = 1;
private static final int STANJE_V_MIROVANJU = 2;
private static final int STANJE_IZVAJA_STORITEV_ZA_STRANKO = 3;
}

```

Interakcija.java

«control» Interakcija
□ bankomat: Bankomat □ pin: int □ stanje: int □ STANJE_BRANJE_KARTICE: int □ STANJE_BRANJE_PIN_STEVILKE: int □ STANJE_IZBIRA_TRANSAKCIJE: int □ STANJE_IZVEDBA_TRANSAKCIJA: int □ STANJE_VRNITEV_KARTICE: int □ STANJE_KONCNO: int
● Interakcija(bankomat: Bankomat) ● kreirajInterakcijo(): void ● nastaviPin(pin: int): void

```
package bankomatIs;

import banka.Kartica;
import banka.Status;
import banka.Potrdilo;
import bankomatIs.fizicno.UporabniskaPlosca;
import bankomatIs.transakcije.Transakcija;

public class Interakcija {

    public Interakcija (Bankomat bankomat) {
        this.bankomat = bankomat;
        stanje = STANJE_BRANJE_KARTICE;
    }

    public void kreirajInterakcijo () {
        Kartica kartica = null;
        Transakcija trenutnaTransakcija = null;
        while (stanje != STANJE_KONCNO) {
            switch (stanje) {
                case STANJE_BRANJE_KARTICE:
                    kartica = bankomat.vrniBralnikKartic().preberiKartico();
                    if (kartica != null)
                        stanje = STANJE_BRANJE_PIN_STEVILKE;
                    else {
                        bankomat.vrniUporabniskoPlosco().prikaziObvestilo("Kartice ni mogoče prebrati");
                        stanje = STANJE_VRNITEV_KARTICE;
                    }
                    break;
                case STANJE_BRANJE_PIN_STEVILKE:
                    try {
                        pin = bankomat.vrniUporabniskoPlosco().zahtevajPIN("Vnesite svojo PIN številko in pritis
                        stanje = STANJE_IZBIRA_TRANSAKCIJE;
                    } catch (UporabniskaPlosca.Preklicano e) {
                        stanje = STANJE_VRNITEV_KARTICE;
                    }
                    break;
                case STANJE_IZBIRA_TRANSAKCIJE:
                    try {
                        trenutnaTransakcija = Transakcija.izvediTransakcijo (bankomat, this, kartica, pin);
                        stanje = STANJE_IZVEDBA_TRANSAKCIJE;
                    } catch (UporabniskaPlosca.Preklicano e) {
                        stanje = STANJE_VRNITEV_KARTICE;
                    }
            }
        }
    }
}
```

```

    }
    break;
case STANJE_IZVEDBA_TRANSAKCIJE:
    try {
        boolean ponovi = trenutnaTransakcija.pozeniTransakciju ();
        if (ponovi)
            stanje = STANJE_IZBIRA_TRANSAKCIJE;
        else
            stanje = STANJE_VRNITEV_KARTICE;
    } catch (Transakcija.KarticaZasezena e) {
        stanje = STANJE_KONCNO;
    }
    break;
case STANJE_VRNITEV_KARTICE:
    bankomat.vrniBralnikKartic().vrniKartico();
    stanje = STANJE_KONCNO;
    break;
}
}
}

```

```

public void nastaviPIN(int pin) {
    this.pin = pin;
}

```

```

private Bankomat bankomat;
private int pin;
private int stanje;
private static final int STANJE_BRANJE_KARTICE = 1;
private static final int STANJE_BRANJE_PIN_STEVILKE = 2;
private static final int STANJE_IZBIRA_TRANSAKCIJE = 3;
private static final int STANJE_IZVEDBA_TRANSAKCIJE = 4;
private static final int STANJE_VRNITEV_KARTICE = 5;
private static final int STANJE_KONCNO = 6;
}

```

BralnikKartic.java

«control» BralnikKartic
□ bankomat: Bankomat
● BralnikKartic(bankomat: Bankomat) ● preberiKartico(): void ● vrniKartico(): void ● obdrziKartic(): void

```
package bankomatIs.physical;

import bankomatIs.Bankomat;
import banka.Kartica;
import simulacija.Simulacija;

public class BralnikKartic {

    public BralnikKartic (Bankomat bankomat) {
        this.bankomat = bankomat;
    }

    public Card preberiKartico () {
        return Simulacija.getInstance().preberiKartico();
    }

    public void vrniKartico () {
        Simulacija.getInstance().vrniKartico();
    }

    public void obdrziKartico () {
        Simulacija.getInstance().obdrziKartico();
    }

    private Bankomat bankomat;
}
```

17.3 Simulacija projekta



17.3.1 Pravila

- **Cilj:**
 - Izdelati vse primere uporabe v čim krajšem času in s čim nižjimi stroški.
- Omejitve časa in stroškov projekta:
 - maksimalno trajanje je **31 tednov**,
 - maksimalen proračun je **100.000 €**.
- Stroški in produktivnost:
 - 1 zaposleni stane **1.000 €/teden**,
 - 1 zaposleni izdela **5 enot primerov uporabe/teden** (ena enota je enaka relativni ceni izdelave primera uporabe),
 - **+ 1 teden/iteracijo** za načrtovanje in režijo.
- Dodatne omejitve:
 - relativne **cene** primerov uporabe (enote) **so konstante**, pravilne in se ne spreminjajo, razen če na to eksplicitno vpliva nepričakovan dogodek,
 - vsak primer uporabe **izdelate v celoti v eni iteraciji**,
 - zaposlene je mogoče dodeliti tudi le delno, kar pomeni, da zaposleni neko vlogo opravlja le del časa, kjer je najmanjša enota za delitev 0,1 (10% časa),
 - načrtovani urnik po izračunu morda ne bo celo število; v tem primeru kot **rezultat** uporabite **najbližje celo število, zaokroženo navzgor**.

17.3.2 Izračun točk simulacije

Če je delo dokončano v okviru načrtovanega proračuna in urnika:

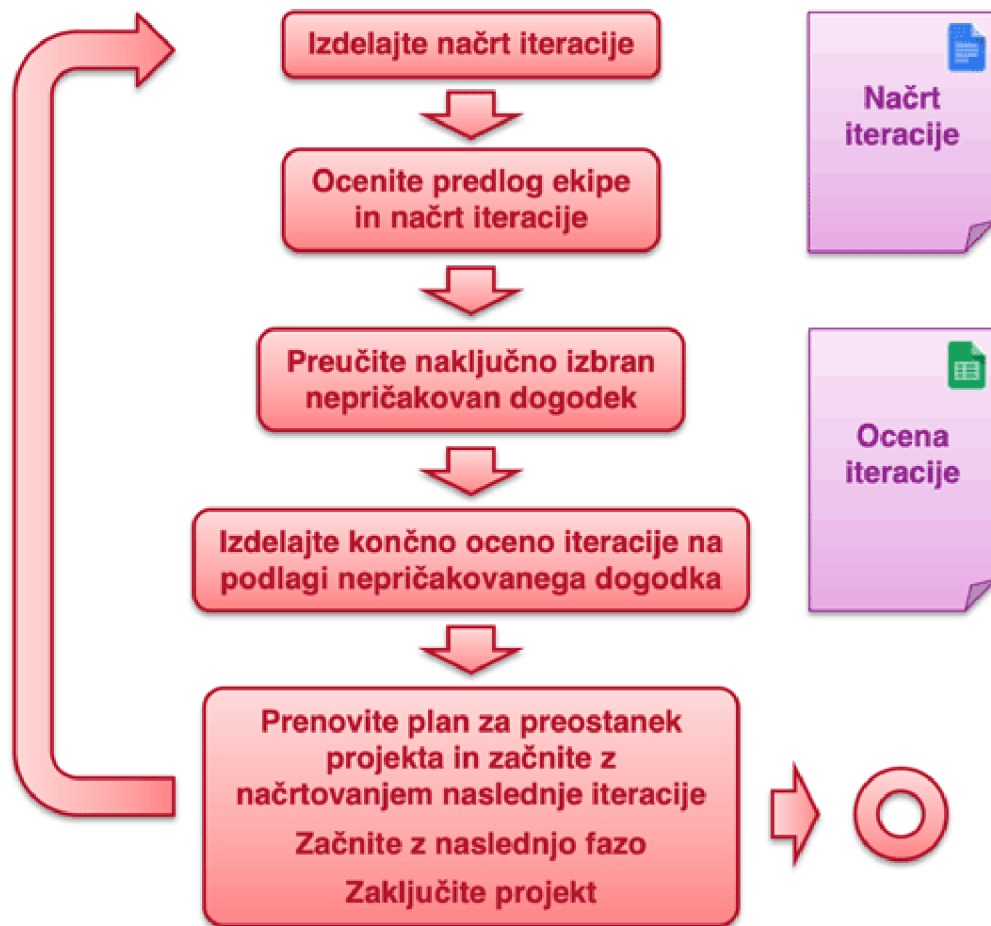
$$\begin{aligned} &= 2.000 \\ &+ (400 \times \text{število tednov pred načrtovanim urnikom}) \\ &+ (200 \times \text{vsakih } 3.000 \text{ € pod načrtovanim proračunom}) \end{aligned}$$

Če je proračun ali urnik prekoračen:

$$\begin{aligned} &= 2.000 \\ &- (200 \times \text{število nedokončanih primerov uporabe}) \\ &- (400 \times \text{število tednov čez rok}) \\ &- (200 \times \text{vsakih } 3.000 \text{ € čez načrtovan proračun}) \end{aligned}$$

17.3.3 Potek simulacije

Najprej izdelate **načrt iteracije** in sledite postopku na sliki 17.62, kjer v korakih izdelujete **oceno iteracije**.



Slika 17.62: Potek simulacije

17.3.3.1 Načrt iteracije

P3.2 - Simulacija projekta - načrt iteracije



17.3.3.2 Ocena iteracije

P3 2 - Simulacija projekta - ocena iteracije



17.3.4 Gradivo

Dokument	Povezava
Načrt iteracije	Google Docs
Ocena iteracije	Google Spreadsheets

17.3.5 Naključni dogodki pri simulaciji

V nadaljevanju je na voljo seznam potencialnih dogodkov po fazi **zbiranja zahtev (ZI)** v tabeli 17.2 in fazi **konstrukcije (K)** v tabeli 17.3. Pri izvedbi simulacije pred oceno iteracije naključno izberite enega izmed dogodkov glede na to, v kateri fazi se nahajate.

Tabela 17.2: Naključni dogodki pri **fazi zbiranja zahtev (ZI)**

Številka dogodka	Dogodek
1	Pri enem od drugih projektov je prišlo do težav. Da bi težave razrešili, si izposodijo arhitekta iz tvojega projekta. Del sistema moraš izdelati brez arhitekta, zato nekateri deli niso zadovoljive arhitekturne kakovosti. Čas te iteracije podaljšaj za en teden.
2	Stranko skrbi predvidena funkcionalnost sistema. Zato zahteva dodaten pregled vseh primerov uporabe, preden se delo nadaljuje. Iteracija se podaljša za en teden.
3	Tvoj projekt ima visoko prioriteto. Tvojemu timu so bili dodeljeni najboljši ljudje v podjetju. V tej iteraciji je bil eden izmed primerov zaključen zelo učinkovito. Znižaj njegovo ceno za 50 %, ko boš računal dejanske stroške in urnik za to iteracijo.
4	Ugotovil si, da si v enem od prejšnjih projektov že izdelal primer uporabe št. 7 . Ta primer uporabe lahko dodaš med že izdelane v tej iteraciji, ne da bi dodajal čas ali denar (če si ta primer uporabe že izdelal, potem bi moral preveriti prej).
5	Nameščeno je bilo novo orodje za testiranje. Usposabljanje preizkuševalcev za delo z orodjem traja dva tedna – dodaj dva koledarska tedna za to iteracijo . Dolgoročno uporaba orodja prihrani čas. Izboljšave na področju testiranja omogočajo, da se čas izdelave vsakega prihodnjega primera uporabe skrajša za pol tedna na primer uporabe.
6	Žal je testiranje pokazalo, da je potrebno enega izmed predhodno izdelanih primerov uporabe izdelati ponovno. Premakni enega izmed že izdelanih primerov uporabe nazaj med neizdelane.
7	Nekaj pomembnih arhitekturnih vprašanj še ni razrešenih. Potrebno je poiskati alternativen pristop. Izberi enega izmed primerov uporabe iz te iteracije za predelavo v naslednji.

Tabela 17.3: Naključni dogodki pri **fazi konstrukcije (K)**

Številka dogodka	Dogodek
1	Sreča! Stranka je zelo zadovoljna z rezultati. Zato da na razpolago več časa in denarja. Proračun se poveča za 3.000 €, urnik pa za en teden.
2	Izdelan je bil prototip, ki je pokazal, kako naj bi izgledal sistem. Stranka je odkrila, da eden od delov sistema nima prave uporabne vrednosti, zato so preklicali naročilo za ta del sistema. Zdaj lahko porabiš več časa za druge dele sistema. Zmanjšaj čas za to iteracijo za en teden.
3	Zaradi nevihte je bilo za dva dni prekinjeno napajanje. V tem času ni mogoče uporabljati računalniške opreme. Ta čas izkoristiš za izvedbo delavnic. Kljub temu boš izgubil en koledarski teden.
4	Zunanja analiza projekta je pokazala, da je kakovost zelo visoka. V tej iteraciji lahko zmanjšaš število enot za enega izmed primerov uporabe za 50 %.
5	Najel si zunanjega svetovalca, da bi opravil del tvojega dela. Izkazalo se je, da svojega dela ni opravil zadovoljivo. Dodaj enega izmed svojih zaposlenih, da bo v tej iteraciji opravil delo. Povečaj porabo v tej iteraciji za dodatno osebo.
6	Težave! Stranka je spremenila svojo organizacijo in določila novo kontaktno osebo za tvoj projekt. Kontaktna oseba je zdaj vpeljana, vendar se je odločila spremeniti nekatere izmed zahtev. Predelaj 2 primera uporabe!
7	Eden od članov ekipe je zbolel. Namesto njega moraš uporabiti drugo osebo. Čas iteracije ostane enak , saj je število oseb, ki delajo na projektu, še vedno enako, vendar pa se v tej iteraciji povečajo stroški za dodatno osebo.

